

Senior .NET Backend Interview Revision Guide

A Curated Collection of 100+ Backend Engineering Concepts
Based on the 100 Days of Backend Engineering series by Srinija Velaga

What's Inside?

- ✓ C# & .NET Fundamentals
- ✓ ASP.NET Core
- ✓ Entity Framework Core
- ✓ SQL Server & Performance Tuning
- ✓ Authentication & Authorization
- ✓ API Design & Security
- ✓ Middleware & Dependency Injection
- ✓ Azure Fundamentals
- ✓ Design Patterns & SOLID Principles
- ✓ Clean Architecture
- ✓ Microservices
- ✓ System Design
- ✓ Production Best Practices
- ✓ Interview-Oriented Concepts

Why this guide?

Instead of searching through 100 individual posts, this guide brings everything together into a structured document for quick learning and interview revision.

Concepts are organized into a logical flow, making it easier to understand how different technologies work together in real-world backend applications.

Original Series by: Srinija Velaga

Compiled & Organized by: Shama Shaik

"Learn consistently. Build confidently. Share knowledge."

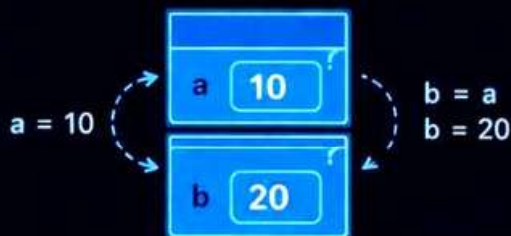
STACK vs HEAP in C#

Understanding how C# stores data can save hours of debugging!

STACK

Value Types – Stores Actual Data

Each variable gets its **own copy**.
Changes **don't affect** the original.

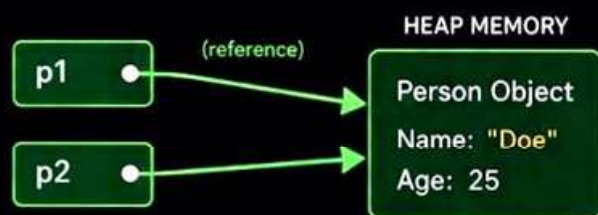


```
int a = 10;
int b = a;
b = 20;
Console.WriteLine(a); // 10
```

HEAP

Reference Types – Stores Objects

Variables store the **reference** (address).
Changes affect the **same object**.



```
class Person { public string Name; public int Age; }

var p1 = new Person { Name = "John", Age = 25 };
var p2 = p1;
p2.Name = "Doe";
Console.WriteLine(p1.Name); // Doe
```



KEY

TAKEAWAY

Value Types → Stack → Independent copies

Reference Types → Heap → Shared data

Small concept,
BIG impact!

Day 1 of 100 Days .NET Backend Journey

Let's grow together!

#SrinijaBuilds

PASS BY VALUE vs PASS BY REFERENCE in C#

Why your variables behave differently in methods



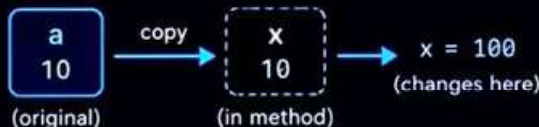
PASS BY VALUE

(Default)

- A copy of the value is passed
- Original value is **NOT** affected
- Safe and predictable

```
void UpdateValue(int x)
{
    x = 100;
}

int a = 10;
UpdateValue(a);
Console.WriteLine(a); // 10
```



Copy behavior



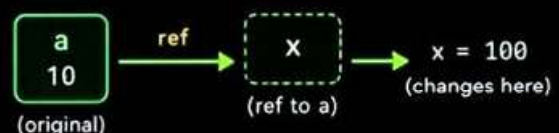
PASS BY REFERENCE

(using ref)

- The actual variable is passed
- Changes reflect outside the method
- Use with caution

```
void UpdateValue(ref int x)
{
    x = 100;
}

int a = 10;
UpdateValue(ref a);
Console.WriteLine(a); // 100
```



Direct modification



IMPORTANT TWIST

Objects are **ALSO** passed by value (but the value is a reference).
You can modify object properties, but not reassign the object itself.



TAKEAWAY: Default = Pass by Value | Use **ref** when you want to modify the original

Day 2 of 100 Days .NET Backend Journey

Let's grow together!

#SrinijaBuilds

EVERYTHING IN C# IS PASS BY VALUE.



Yes, even objects. Here's why it still feels like reference!

In C#, when you pass a variable to a method, a **COPY** of its value is passed. For objects, the value being passed is the **REFERENCE** (address) to the object.



MODIFYING PROPERTIES WORKS

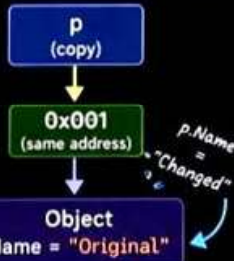
(Because the copy points to the same object)

```
class Person { public string Name; }
void Update(Person p)
{
    p.Name = "Changed";
}
var person = new Person { Name = "Original" };
Update(person);
Console.WriteLine(person.Name); // Changed
```

Before calling Update(person)



Inside method (p = copy)



Both point to the **same object**, so the change is visible outside.



REASSIGNING THE OBJECT DOES NOT WORK

(Because only the copy of the reference is replaced)

```
void Update(Person p)
{
    p = new Person { Name = "New" }; // reassigns local copy
}
var person = new Person { Name = "Original" };
Update(person);
Console.WriteLine(person.Name); // Original
```

Before calling Update(person)



Inside method (p = copy)



Only p changed.
Original reference
(person) is
unaffected.

person still points to the **original object**.



VISUAL SUMMARY

Copies are passed...
what the copy points to
is what matters!

variable
(copy)

address
(value)

OBJECT
(in HEAP)

Modify inside?

Reassign inside?



KEY TAKEAWAY

- Everything is Pass by Value
- Objects feel like reference because the value is an address
- Modify properties: **Works**
- Reassign object: **Doesn't work** (without ref)

The more you understand how **memory works**, the better you write and debug.

#SrinijaBuilds

#100DaysOfCode

CLASS vs STRUCT vs RECORD

Choosing the right type makes your code *faster, safer, and cleaner*.

C# gives us three powerful ways to model data. Each has a purpose.
Let's understand when to use what.



CLASS

Reference Type

A class is a blueprint for creating objects. It's stored on the **HEAP** and you work with a **reference** to that object.

KEY POINTS

- ✓ Stored in Heap
- ✓ Reference is copied
- ✓ Mutable by default
- ✓ Supports inheritance

BEST FOR

Business logic, services, entities, anything with identity and behavior.

```
public class User
{
    public string Name { get; set; }
    public int Age { get; set; }
}

var user1 = new User { Name = "Alice", Age = 25 };
var user2 = user1; // reference copy
user2.Name = "Bob";
Console.WriteLine(user1.Name); // Bob
```



Both reference the same object



STRUCT

Value Type

A struct stores its data directly (usually on the **STACK**). When you pass it around, the entire value is copied.

KEY POINTS

- ✓ Stored on Stack (or inline)
- ✓ Value is copied
- ✓ Great performance for small data
- ✓ Cannot have a parameterless constructor

BEST FOR

Small, lightweight data (e.g., Point, Money), performance-critical scenarios.

```
public struct Point
{
    public int X;
    public int Y;
}

var p1 = new Point { X = 10, Y = 20 };
var p2 = p1; // value copy
p2.X = 99;
Console.WriteLine(p1.X); // 10
```



Two independent copies



RECORD

Reference Type (Immutable by default)

A record is a modern way to create **immutable** data models. It shines for DTOs and data shaping. It provides **value-based equality**.

KEY POINTS

- ✓ Stored in Heap
- ✓ Immutable by default (with init-only)
- ✓ Value-based equality (compares by content)
- ✓ Great for DTOs and simple data carriers

BEST FOR

DTOs, API request/response models, and data that shouldn't change.

```
public record UserDto(string Name, int Age);

var u1 = new UserDto("Alice", 25);
var u2 = new UserDto("Alice", 25);
Console.WriteLine(u1 == u2); // True
// Value-based equality!

// With init-only properties
public record ProductDto
{
    public string Name { get; init; }
    public decimal Price { get; init; }
}
```



Different objects, same content → equal

QUICK COMPARISON



CLASS



STRUCT



RECORD

	CLASS	STRUCT	RECORD
Type	Reference	Value	Reference (Immutable)
Where stored?	Heap	Stack (or inline)	Heap
Copy behavior	Reference copy	Value copy (full copy)	Reference copy
Mutability	Mutable	Mutable (fields)	Immutable (by default) (init-only)
Equality	Reference equality (same object?)	Value equality (compares fields)	Value equality (compares content)
Best for	Business logic, Entities	Small data, Performance	DTOs, API models, Data
Performance	Good	Best (for small data)	Good

WHEN I USE WHAT



CLASS

When the object has identity, behavior, and will change.



STRUCT

When the data is small, simple, and performance matters.



RECORD

When I need an immutable data model with value-based equality.



TAKEAWAY: Use the right tool for the job. It keeps your code fast, predictable, and easy to maintain.

Which one would you pick for a high-traffic API request model? Class, Struct, or Record? Let me know!

#SrinijaBuilds

#100DaysOfCode



WHY IMMUTABILITY CAN SAVE YOU FROM HIDDEN BUGS



*Changing data you didn't mean to change is a **nightmare**.*

Immutability means once data is created, it can't be changed.
This small rule prevents big problems.



PREDICTABLE CODE

Data stays the same.
No side effects.
Easier to reason about.



BUG RESISTANT

Accidental changes?
Blocked.
Your data stays safe.



BETTER FOR CONCURRENCY

Immutable data is
thread-safe by default.
Perfect for high-traffic APIs.



THE PROBLEM (Mutable Data)

When data can change, unexpected bugs appear.

```
public class Order
{
    public int Id { get; set; }
    public decimal Amount { get; set; }
}

var order = new Order { Id = 1, Amount = 100 };
Process(order);
Log(order); // Oops! Amount changed somewhere
```

If something
modifies 'order'
inside a method,
you may never
notice!



⚠️ Hard to track. Easy to break.



THE SOLUTION (Immutable Data)

With immutable data, changes are intentional and safe.

```
public record Order(int Id, decimal Amount);
{
    var order = new Order(1, 100);
    var updated = order with { Amount = 150 }; // new object
    Process(updated);
    Log(order); // Original is unchanged
}
```

You can't modify.
You can only create
a new one.



✅ Original stays unchanged. Bugs stay away.

REAL WORLD EXAMPLE

In a high-traffic API, multiple requests may read the same data.
If one request changes it by mistake, others get corrupted data.
Immutable objects = clean, isolated, and safe.



HOW TO GET IMMUTABILITY IN C#

- Use record (best for data)
- Use init-only properties
- Use with expression (records)
- Avoid set; when not needed

```
public record User(string Name, int Age);
var u1 = new User("Alice", 25);
var u2 = u1 with { Age = 26 }; // new object
```



WHEN TO USE IT

- ✔ DTOs (API requests/responses)
- ✔ Configuration objects
- ✔ Domain events
- ✔ Any data that should not change



WHEN NOT TO USE IT

- ✗ Large objects changed frequently
- ✗ Performance-critical hot paths
- ✗ When mutability simplifies logic



TAKEAWAY: Immutable data = Predictable code + Fewer bugs + Safer systems. Small habit, huge impact.

Write code that doesn't surprise you (or your future self).

#SrinijaBuilds

#100DaysOfCode



ASYNC/AWAIT IN C#



Why your API becomes **slow** sometimes (and how to **fix** it)

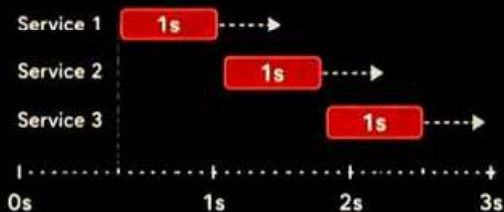
async/await makes your code **NON-BLOCKING**.
It does **NOT** make it automatically parallel.



SEQUENTIAL AWAITS (SLOW)

Runs one after another

```
public async Task GetDataAsync()
{
    var r1 = await CallService1Async();
    var r2 = await CallService2Async();
    var r3 = await CallService3Async();
    return (r1, r2, r3);
}
```



Total time: ~3 seconds



PARALLEL EXECUTION (FAST)

Runs all together

```
public async Task GetDataAsync()
{
    var t1 = CallService1Async();
    var t2 = CallService2Async();
    var t3 = CallService3Async();
    await Task.WhenAll(t1, t2, t3);
    return (await t1, await t2, await t3);
}
```



Total time: ~1 second



WHAT'S HAPPENING?

await → Non-blocking
(frees the thread while waiting)

Task.WhenAll() → Parallel
(all tasks run at the same time)

Result → Blocks the thread
(can cause deadlocks)



COMMON MISTAKES

- ✗ Awaiting calls one by one (loses performance)
- ✗ Using `.Result` or `.Wait()` (can deadlock your app)
- ✗ Not understanding the execution flow



REAL-WORLD IMPACT

- ✓ Faster APIs
- ✓ Better scalability
- ✓ Efficient resource usage
- ✓ Happier users 😊



BEHIND THE SCENES: WHAT REALLY HAPPENS



Method starts on a thread



await hits an I/O task (e.g., HTTP call, DB)



Thread is returned to the pool (not blocked)



When the I/O completes, the continuation runs on any available thread



Your code resumes after the await



QUICK TIP

If the tasks are independent, start them first, then await with **Task.WhenAll()**.



WHEN TO USE WHAT

await

Use for dependent operations
(one needs the result of the previous one)



Task.WhenAll

Use for independent operations
(no dependencies)



EXAMPLE SCENARIO

Your API calls 3 services (User, Order, Payment).
Run them in parallel → **3x faster responses!**



TAKEAWAY:

Async/await keeps your app responsive. Parallel execution makes it **FAST**.
Use the right tool for the right job.



WHY **.Result** AND **.Wait()** CAN **BREAK** YOUR APPLICATION



*They look harmless. They compile. But they can **freeze** your app.*

.Result and **.Wait()** block the current thread until the async operation completes.
In the wrong context, this leads to **deadlocks** and **poor performance**.

THE PROBLEM: THREAD BLOCKING

When you call **.Result** or **.Wait()**, the current thread is **BLOCKED**. It can't do any other work.

```
public string GetDataSync()
{
    var data = GetDataAsync().Result; // BLOCKS!
    return data;
}
```

Looks Easy, Right?



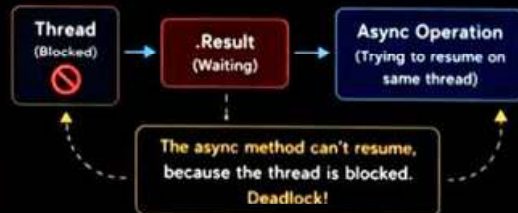
The thread is stuck here, waiting...

⚠ Other work that needed this thread? Also stuck.

THE REAL DANGER: DEADLOCK

In environments with a **SynchronizationContext** (like ASP.NET Classic, WinForms, WPF), a deadlock can occur.

```
public string GetDataSync()
{
    return GetDataAsync().Result; // Deadlock risk!
}
```



WHERE IT HAPPENS

- ASP.NET (Classic)
- WinForms / WPF
- UI Thread
- Any custom SynchronizationContext

SYMPTOMS

- ✓ App freezes
- ✓ High CPU (sometimes)
- ✓ Request timeouts
- ✓ Threads get exhausted

THE RIGHT WAY: AWAIT ALL THE WAY

Let async methods be async. Don't block the thread. This keeps your app responsive and scalable.

```
public async Task<string> GetDataAsync()
{
    var data = await _httpClient.GetStringAsync("api/data");
    return data;
}
```



EXAMPLE: THE DIFFERENCE

BAD (Blocks Thread)

```
var result =
  GetUserAsync().Result;
return result;
```

⌚ Takes longer, blocks other work

GOOD (Non-Blocking)

```
var result =
  await GetUserAsync();
return result;
```

⚡ Faster, scalable, handles more requests

QUICK RULES

- ✓ Use **async** all the way
- ✓ Use **await** for async calls
- ✓ Use **Task.WhenAll** for parallel calls
- ✗ Don't use **.Result** or **.Wait()**
- ✗ Don't block async code
- ✗ Don't mix sync over async

REAL-WORLD IMPACT

- ✓ Prevents deadlocks
- ✓ Better throughput
- ✓ Lower latency
- ✓ Scales under load
- ✓ Happier users 😊

ONE MORE THING

In ASP.NET Core (no custom SynchronizationContext), **.Result** usually won't deadlock... but it still blocks threads, hurts performance, and reduces scalability.



TAKEAWAY:

.Result and **.Wait()** might seem convenient, but they can freeze your app.
Use **await**, keep threads free, and your API will scale beautifully.

Understand the why, not just the how. That's how you level up.

#SrinijaBuilds #100DaysOfCode



DEPENDENCY INJECTION IN .NET

WHAT ACTUALLY HAPPENS BEHIND THE SCENES?



DI is a design pattern where objects **receive** their dependencies from an **external source** instead of creating them.



LOOSE COUPLING

Classes don't depend on concrete implementations.



EASY TESTING

Dependencies can be mocked easily.



CLEAN ARCHITECTURE

Promotes SOLID principles and maintainability.



BETTER SCALABILITY

Makes applications easier to extend and scale.

WHAT IT LOOKS LIKE (HIGH LEVEL)

```
public class UserController : ControllerBase
{
    private readonly IUserService _service;

    public UserController(IUserService service)
    {
        _service = service; // DI happens here!
    }
}
```



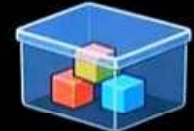
The controller doesn't create the service. It just asks for it.

WHAT HAPPENS BEHIND THE SCENES



```
builder.Services.AddScoped<IUserService, UserService>();
// Interface           // Implementation
```

.NET's DI Container holds this mapping. When `UserController` is created, it asks the container for `IUserService`, and the container provides `UserService`.



SERVICE LIFETIMES (HOW LONG AN INSTANCE LIVES)



SCOPED

One instance per request. Perfect for database contexts, unit of work, etc.



TRANSIENT

A new instance every time it's requested. Lightweight, stateless services.



SINGLETON

One instance for the entire application lifetime. Use for stateless, thread-safe services only.



LET'S SEE A FULL EXAMPLE

```
builder.Services.AddScoped<IUserService, UserService>();

public interface IUserService
{
    Task<User> GetUserAsync(int id);
}

public class UserService : IUserService
{
    public Task<User> GetUserAsync(int id) => ...
}
```

Registered in the DI container



COMMON MISTAKES

- ✗ Registering the wrong lifetime (e.g., Singleton for `DbContext`).
- ✗ Using "new" to create dependencies manually.
- ✗ Forgetting to register a service (leads to runtime error).



REAL-WORLD IMPACT

- ✓ Makes your code testable.
- ✓ Keeps components independent.
- ✓ Easier to change implementations without touching consumers.
- ✓ Essential for clean architecture.



TAKEAWAY: You focus on what your classes need, not how to create it. The container handles the rest.

Good architecture starts with loose coupling. DI is your superpower.

#SrinijaBuilds #100DaysOfCode



MIDDLEWARE IN .NET

HOW YOUR REQUEST ACTUALLY FLOWS



It's not magic. It's a pipeline.



Middleware is a series of components in the request pipeline. Each component can process the request, do some work, and pass it to the next one.



</> EXAMPLE: REGISTERING MIDDLEWARE

```

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.UseExceptionHandler(); // Handles errors
app.UseHttpsRedirection(); // Redirects to HTTPS
app.UseStaticFiles(); // Serves static files
app.UseAuthentication(); // Authenticates user
app.UseAuthorization(); // Authorizes user
app.MapControllers(); // Maps endpoints

app.Run();
  
```

WHAT MIDDLEWARE CAN DO

- Logging**
Track requests and responses.
- Authentication**
Validate user identity.
- Authorization**
Check user permissions.
- Error Handling**
Catch and handle exceptions.
- Routing**
Map requests to endpoints.

IMPORTANT: ORDER MATTERS

The order you register middleware changes the behavior of your app.

```

app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
  
```

- Wrong Order** → Auth may fail
- Correct Order** → Works as expected



REAL-WORLD EXAMPLE

A request to /api/users might flow:

- Logging** → Log the request
- Authentication** → Validate token
- Authorization** → Check roles
- Routing** → Match to UsersController
- Controller** → Return users



CUSTOM MIDDLEWARE

```

public class RequestTimeMiddleware
{
    private readonly RequestDelegate _next;

    public RequestTimeMiddleware(RequestDelegate next)
    { _next = next; }

    public async Task Invoke(HttpContext context)
    {
        var start = DateTime.UtcNow;
        await _next(context);
        var duration = DateTime.UtcNow - start;
        Console.WriteLine("Request took {0}ms",
            (duration.TotalMilliseconds) ms);
    }
}

app.UseMiddleware<RequestTimeMiddleware>();
  
```



QUICK TIPS

- Keep middleware small and focused.
- Always call the next middleware (`_next`).
- Use built-in middleware when possible.
- Add custom middleware for cross-cutting concerns.



TAKEAWAY

Middleware is the heart of the request pipeline in ASP.NET Core.

Order defines behavior. Understand the flow, build better APIs.

Understand the flow. Control the pipeline. Build better applications.

#SrinijaBuilds

#100DaysOfCode

#DotNet

#CSharp

FILTERS vs MIDDLEWARE

WHEN TO USE WHAT?



Every HTTP request passes through the **Middleware Pipeline** before it reaches your controller.

• THE REQUEST → RESPONSE PIPELINE •



Filters

Filters are part of the **MVC pipeline** and only run around controller actions.

- Runs **INSIDE** the MVC pipeline
- Closer to the **ACTION**
- Often applied **PER ACTION**

VS



Middleware

Middleware is part of the **global pipeline** and runs for **EVERY** request.

- Part of the **REQUEST PIPELINE**
- Runs **BEFORE CONTROLLER**
- Works **GLOBALLY**



• KEY DIFFERENCES



Filters

```
public class LogFilter : IActionFilter {
    public void OnActionExecuting(...) {
    public void OnActionExecuted(...) {
    }
}
```

- ✓ Runs around MVC actions
- ✓ Closer to controller
- ✓ Fine-grained control



MIDDLEWARE



Middleware

```
app.UseLogging();
app.UseAuthentication();
app.UseAuthorization();
}
```

- ✓ Runs for **ALL requests**
- ✓ In request pipeline (before controller)
- ✓ Handles cross-cutting concerns

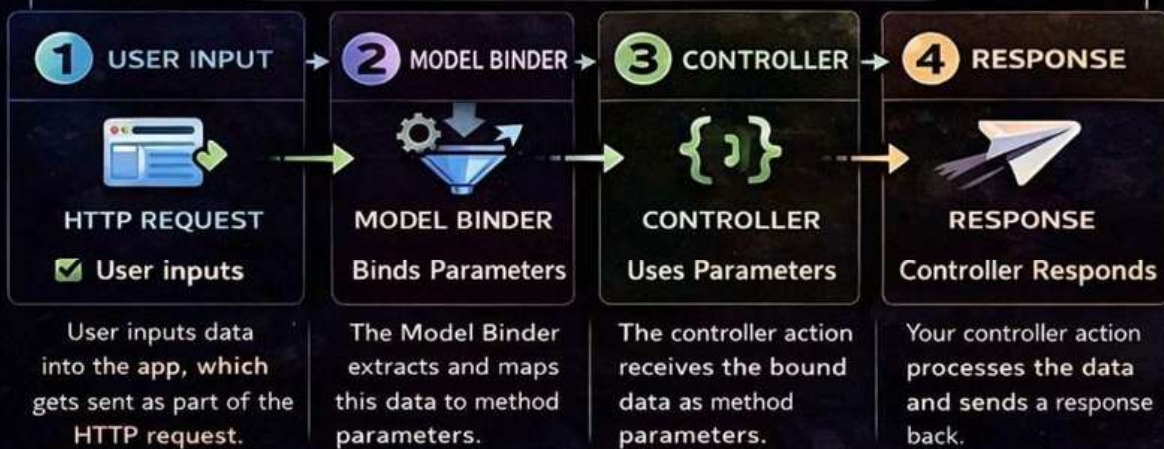
MODEL BINDING IN .NET

HOW DATA REACHES YOUR API PARAMETERS



Ever wondered how data gets from the HTTP request to your controller action?

THE PROCESS OF MODEL BINDING



SOURCES OF DATA



{+} CONTROLLER ACTION

```
[HttpGet("/users/{id}")]  
public IActionResult GetUser(string name, int id)
```

TAKEAWAY!

Model Binding is what automatically converts incoming HTTP data into your controller parameters.

- ✓ Maps incoming data to params
- ✓ No manual parsing needed
- ✓ Works with: Query, Route, Body, etc. | Simplifies controller logic

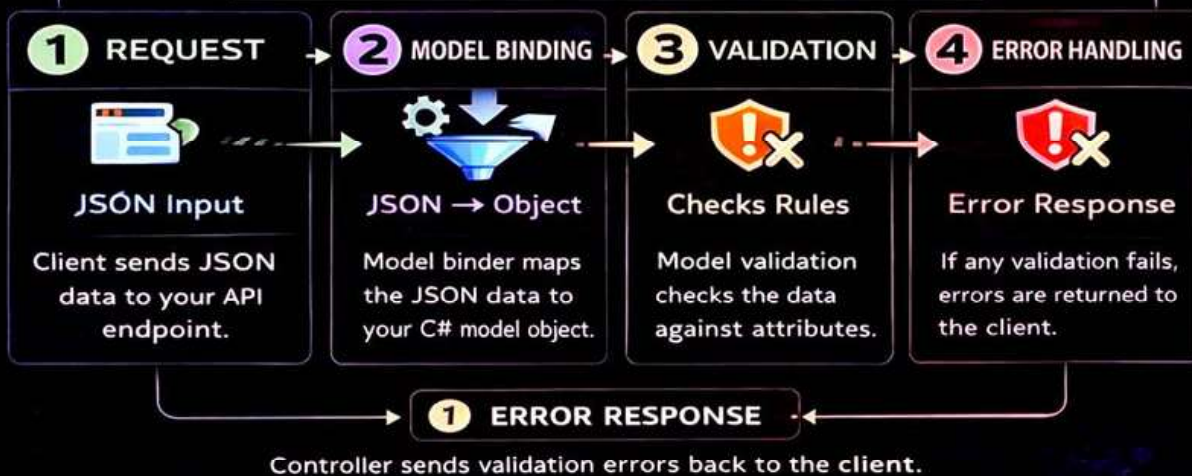
MODEL VALIDATION

ENSURE YOUR API RECEIVES ONLY VALID DATA



How do you make sure requests contain the right data before they hit your controller?

HOW DOES MODEL VALIDATION WORK?



EXAMPLE CODE

```
public class CreateUserRequest {  
    [Required]  
    public string Name { get; set; }  
    [Required]  
    [EmailAddress] public Email { get; set; }  
}
```

ATTRIBUTES YOU CAN USE

- | | |
|---------------------|---|
| ✓ [Required] | ✓ [Required] Checks if a field to be filled |
| ✓ [EmailAddress] | ✓ [EmailAddress] Checks if input is a valid email |
| ✓ [Range (18, 100)] | ✓ [StringLength(50)] Limits max length of strings |

TAKEAWAY!

Model Validation = Automatic input validation for controllers.

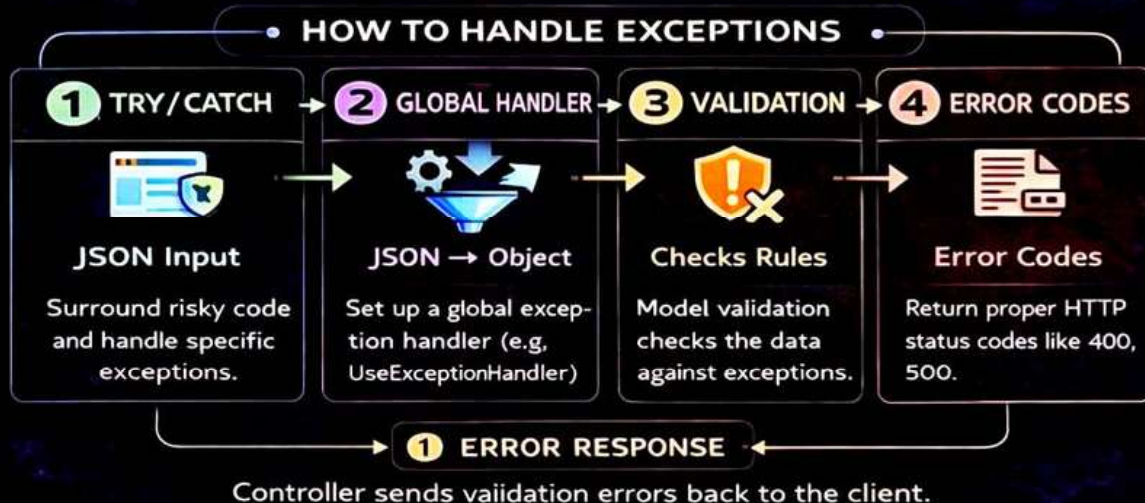
- ✓ Guards your endpoints
- ✓ Returns detailed error messages

EXCEPTION HANDLING

HOW TO HANDLE ERRORS PROPERLY



What happens when your code goes wrong?



EXAMPLE CODE

```

[HttpGet("/users/{id}")]
public IActionResult GetUser(int id) {           // Risky code
    try
        var user = FindUserId(id);           // Risky code
        return Ok(user);
    } catch (Exception ex) {
        // Log the exception
        return StatusCode(500, new { ex.Message });
    }
}
    
```

TRY / CATCH BEST PRACTICES

- ✓ Be **SPECIFIC** with exceptions
- ✓ **USE** logging to track errors
- ✓ **RETURN** user-friendly messages
- ✓ **AVOID** swallow and ignore

🏆 TAKEAWAY!

Exception Handling = Managing errors gracefully

- ✓ Keep your app stable
- ✓ Ensures detailed error
- ✓ Reduces bugs

LOGGING IN .NET



HOW TO SEE WHAT'S HAPPENING INSIDE YOUR APP

Ever wondered what's going on IN REAL-TIME as your app runs?

LOGGING shows you the inner workings of your .NET application.

WHAT TO LOG



Events

Significant actions or milestones



Errors

Exceptions and failures



Health

System performance metrics

LOG LEVELS



TRACE

Fine details



DEBUG

Debug information



INFORMATION

General events



WARNING

Potential issues



ERROR

Failures

EXAMPLE CODE

```
[HttpPost]
public IActionResult CreateUser(CreateUserRequest model)
{
    try
    {
        _logger.LogInformation("Creating a new user...");
    }
    if (!ModelState.IsValid)
    {
        _logger.LogWarning("Validation failed.");
    }
    return BadRequest(500, new { ex.Message });
}
```

```
try
{
    _ Database operation here
}
catch (Exception ex)
{
    _logger.LogError(ex, "Error creating user!");
    return StatusCode(500, "An error occurred.");
}
```

BEST PRACTICES

- ✓ Use right log LEVEL
- ✓ Log with CONTEXT
- ✓ DON'T log sensitive info
- ✓ Use a centralized **LOGGER**.



TAKEAWAY!

Logging = Visibility & insight instantly

- ✓ Track down issues
- ✓ Monitor behavior
- ✓ Keeps your app healthy



CONFIGURATION IN .NET

HOW YOUR APP BEHAVES IN DIFFERENT ENVIRONMENTS

**SCENARIO**

You have multiple environments:
• Development • QA • Production

Should all use the SAME database?

**SOLUTION**

Use Configuration to manage settings
outside your code.

Same code, different behavior!

WHAT IS CONFIGURATION?

Configuration is the process of managing application settings outside the code so that your app can behave differently in different environments.

1. APPSETTINGS.JSON (BASE FILE)

```
{
  "ConnectionStrings": {
    "Default": "Server=.;Database=AppDb;"
  },
  "ApiSettings": {
    "BaseUrl": "https://api.myapp.com",
    "Timeout": 30
  },
  "Logging": {
    "LogLevel": "Information"
  }
}
```



Holds default configuration values.

2. ENVIRONMENT-SPECIFIC FILES

appsettings.Development.json

→ For local development



appsettings.QA.json

→ For QA environment



appsettings.Production.json

→ For production environment



.NET automatically loads the file based on the environment.

3. ACCESS IN CODE

```
public class MyService
{
    private readonly IConfiguration _config;

    public MyService(IConfiguration config)
    {
        _config = config;
    }

    public string GetConnectionString()
    {
        return _config.GetConnectionString("Default");
    }
}
```



Use IConfiguration to read values
anywhere in your application.

4. HIERARCHY (ORDER OF PRIORITY)

appsettings.json
(Base file)



appsettings.(Environment).json
(Overrides base)



Environment Variables
(Overrides JSON)



User Secrets / Key Vault
(Highest priority)

**BEST PRACTICES**

- ✓ Never hardcode secrets in code
- ✓ Use environment variables for sensitive data
- ✓ Use User Secrets (dev) or Azure Key Vault (prod)
- ✓ Keep configuration values organized and meaningful
- ✓ Validate configuration on startup

**MISTAKE I MADE**

Hardcoding connection strings and secrets.

It worked locally, but caused:

- Security risks
- Deployment issues
- Hard to manage in multiple environments

**TAKEAWAY**

Code stays the same, but configuration changes behavior.

Configuration = **Flexibility, Security, and Scalability**

**SCENARIO FOR YOU**

If your app moves to production,
would you hardcode configs or
manage them properly?



CACHING IN .NET

HOW TO BOOST YOUR API PERFORMANCE!

appsettings.json



Real Scenario: You have Dev, QA, and Production environments. Should all of them use **SAME database**? ❌
Of course not! That's where **Configuration** comes in.

WHAT IS CACHING? ⚡

Caching temporarily **stores data** so that **future requests** can be served much faster.



Blazing Fast Responses



Reduced Server Load



Cost Efficient



That's what **Caching** is for.

EXAMPLE CODE

```
public class UsersController
{
    private readonly IMemoryCache
        _cache;
    public UsersController (IMemoryCache cache)
    {
        _cache = cache,
    }
    [HttpGet("/user/{id}")]
    public IActionResult GetUser(int id)
    {
        // Check if data is cached
        if (_cache.TryGetValue("User::id", out
            _User user)
        } return Ok(user) : Return cached data
    }
}
```

EXAMPLE: appsettings.json

```
{
  public class UsersController : ControllerBase
  {
    private readonly IMemoryCache cache;
    public UsersController(IMemoryCache cache)
    {
        _cache = cache,
    }
    [HttpGet("/user/{id}")]
    {
        public IActionResult GetUser (int id)
        {
            // Check if data is cached
            if (_cache.TryGetValue("User::id", out
                User user)
            } return Ok(user) : Return cached data
        }
        user = FetchUserFromDatabase(id);
        _cache.Set("User::id", user, TimeSpan.FromMinute(5));
    }
}
```

BEST PRACTICES



1. DON'T HARDCODE DATA

- Fastest way to store and re-use data in server memory.



2. USE ENVIRONMENT VARS,

- Fewer database and API calls



3. INVALIDATE and REFRESH

- Saves computing power



TAKEAWAY! ♦

Caching = Speed & Efficiency

Your code stays the same, but configuration makes it work

Think About It:

if your app moves to a API or a super fast API?

LINQ in .NET

HOW TO WRITE CLEANER QUERIES!



⚡ LINQ (Language Integrated Query)



Sick of **for-loops** when querying data?

```
List<tring> topNames = new()
foreach(var person in people)
{
    if(person.Age > 21)
        topNames.Add(person.Name);
}
```

Want a better, cleaner way? That's LINQ!

WHAT IS LINQ? ⚡

LINQ in .NET allows you to write **queries** directly in C# using simple, readable syntax to filter, sort, and manipulate data.



Fluent Queries

A declarative style to express what you need.



Cleaner Queries

Replaces complex loops with clean expressions.



Less Code

Condenses operations into fewer lines.



Find Names Of People Over 21 (Alphabetically ordered).

```
var topNames = people
    . Where (p > p.Age > 21)
    . OrderBy (p > p.Name)
    . Select (p > p.Name)
ToList();
```

RESULT: John, Alice, Bob

◆ LINQ ROLES

◆ WHERE

- ◆ Filter items based on a condition

SELECT

Project elements into new form

ORDERBY

Sort by a specified field
Groups items by key



JOIN

Merges two collections into one

ANY

Checks if any item meets a condition

SUM

Aggregates a collection

FIRST

Gets the first element

◆ BEST PRACTICES ◆

- ✓ Use the **RIGHT** operator for job
- ✓ **AVOID** n>t problems...
- ✓ Don't select **MORE** than needed



TAKEAWAY! ◆

LINQ = Modern Data Querying in C#

- Cleaner, simpler code ✓ Powerful operators

Think About It: ?

Would you prefer long, complex loops, or legant LINQ queries?

COLLECTIONS

in C#

List vs Dictionary vs HashSet



⚡ Unsure when to use **List**, **Dictionary**, or **HashSet**?
Understanding their differences is **KEY**!

1 2 3	LIST	🔑 DICTIONARY	📋 HASHSET
	General Idea: Ordered collection for storing items	General Idea: Key-value pairs for fast lookups	General Idea: Collection of UNIQUE values
	IDEAL WHEN: ✓ Need sequential access ✓ Allow duplicates ✓ Order matters	IDEAL WHEN: ✓ Quickly find by key ✓ Unordered collection ✓ Keys MUST be unique	IDEAL WHEN: ✓ Values MUST be unique ✓ Unordered, no duplicates ✓ Fast Union/Intersection
	Example code: <pre>var numbers = new List<int> { 1, 2, 3, 4, 5 }; numbers.Add(10); foreach (var n in numbers) { }</pre>	Example code: <pre>var userAges = new Dictionary<string, int> { { "Alice", 30 }, { "Bob", 25 }, } Console.WriteLine(userAges["Alice"]);</pre>	Example code: <pre>var uniqueNumbers = new HashSet<int> { 1, 2, 3, 4, 5, }; uniqueNumbers.Add(10); }</pre>
	INSERT: FAST 	LOOKUP: FAST 	CONTAINS: FAST

🏆 KEY DIFFERENCES		
List	Dictionary	HashSet
✓ Ordered. allows duplicates allowed	✓ Fast lookups by key	✓ Enforces unique values

⚡ TAKEAWAY!

✓ LINQ = Clean + expressive data queries

Think About it: ?

Which collection are you using the most in CA?

IEnumerable vs IQueryable

IN .NET

HIDDEN PERFORMANCE DIFFERENCE!

Do you know what's happening **BEHIND** the scenes?

Let's start with a real scenario 🔍

UNDERSTANDING THE DIFFERENCE

IEnumerable

- **Enumerable<User>**
Ordered collection for storing items
- **IDEAL WHEN:**
 - ✓ Order matters
 - ✓ Sequential access

IQueryable

- **IQueryable<User>**
Key-value pairs for fast lookups
- **IDEAL WHEN:**
 - ✓ Need fast search by **key**
 - ✓ Unique keys

IQueryable

- **IQueryable<User>**
Executes **QUERY** on **DATABASE**
- **USE WHEN:**
 - ✓ Need fast search by **key**
 - ✓ Unique keys

Looks similar, right? **NOT QUITE!**

UNDERSTANDING THE DIFFERENCE



IEnumerable

Pulls records into MEMORY first

```
var numbers = new List<int>
    .Where(u > u.Age > 21);
    .Select(u > o.Name);
}
```

IN MEMORY query



IQueryable

Executes QUERY on DATABASE

```
var names = users
    .Where(u => u.Age > 21)
    .Select(u > o.Name);
}
```

IN DATABASE query

COMPARISON:

IEnumerable			IQueryable
⌚ It's small, in-memory data	🔍	⚡	📄 ONE QUERY
🌟 Filters reparse	MULTIPLE	LOOKUP	🗄️ Large, database queries



BEST PRACTICES: ✨

IEnumerable → Memory-based
IQueryable → Database-based

Think About It: ?

Using **IEnumerable**,
for heavy database
queries—Consider
IQueryable!

ASYNCHRONOUS STREAMS IN .NET

HOW TO HANDLE LARGE DATA EFFICIENTLY!

- Working with **large DB data**?
- Hit the Database **ONCE...** or **one row at a time**?

IEnumerable<T>

- Loads **ENTIRE** dataset into memory first.
- High MEMORY** usage.
- Delays and **BLOCKING**



ALL AT - ONCE

```
// Fetch everything at once
var items = context.Items.ToList();
foreach(var item in items)
{ await ProcessAsync(item);
}
```



IAsyncEnumerable<T>

- Streams **one ROW** at a time from the DB.
- Low MEMORY** usage.
- MINIMAL** waiting time.



ONE-BY-ONE

```
// Fetch one-by-one
await foreach(var item in context.Items)
{ await ProcessAsync(item);
}
```



COMPARISON:

IEnumerable<T>	IAsyncEnumerable<T>
MEMORY USAGE HIGH, loads everything	EXECUTION ✓ ALL AT ONCE
DATA fits in memory 	AS DATA IS NEEDED

BEST PRACTICES:

- Return **IAsyncEnumerable<T>** for large datasets
- Release resources **FAST** after processing



TAKEAWAY!

IEnumerable<T> → All at once, memory-heavy
IAsyncEnumerable<T> → One-by-one, memory light



Think About It?

Massive O9-query?
 Would you process everything at once, or stream it?

DAY 21 OF 100 DAYS .NET BACKEND JOURNEY

SOLID PRINCIPLES

WRITE CODE THAT **SCALES**, NOT BREAKS!



REAL SCENARIO:

You update one feature...
and suddenly 3 other things break!

SOLUTION:

Follow SOLID principles to build
clean, maintainable and **scalable** code.



S – SINGLE RESPONSIBILITY PRINCIPLE

One class = One responsibility

BAD EXAMPLE:

UserService handles
DB + Email + Validation



GOOD EXAMPLE:

Separate services for each



Why? Easier to change and test.



O – OPEN / CLOSED PRINCIPLE

Open for extension,
closed for modification

BAD EXAMPLE:

Modify existing code
every time you add new behavior.

```
class Discount {
    public decimal GetDiscount(
        string type) {
        if(type == "Regular") ...
        else if(type == "Premium") ...
        else if(type == "VIP") ...
    }
}
```

GOOD EXAMPLE:

Extend without modifying existing code.

```
interface IDiscount {
    decimal GetDiscount();
}
class PremiumDiscount : IDiscount {
    public decimal GetDiscount(){ ... }
}
```

Why? Easy to add new features.



L – LISKOV SUBSTITUTION PRINCIPLE

Derived class should behave
like base class

BAD EXAMPLE:

Derived class breaks
expected behavior.

```
class Bird {
    public virtual void Fly() { ... }
}
class Ostrich : Bird {
    public override void Fly() {
        throw new Exception("Cannot fly");
    }
}
```

GOOD EXAMPLE:

Derived class respects base behavior.

```
class Bird { public virtual void Move() { ... } }
class Ostrich : Bird {
    public override void Move() { ... }
}
```

Why? Code becomes predictable.



I – INTERFACE SEGREGATION PRINCIPLE

Don't force classes to implement
methods they don't use.

BAD EXAMPLE:

Big interface with methods
that some classes don't need.

```
interface IWorker {
    void Work();
    void Eat();
    void Sleep();
    void Swim();
}
```



GOOD EXAMPLE:

Small and specific interfaces.

```
interface IWork {
    void Work();
}
```

```
interface ISwim {
    void Swim();
}
```

Why? Cleaner and easier to maintain.



D – DEPENDENCY INVERSION PRINCIPLE

Depend on abstractions,
not on concrete classes.

BAD EXAMPLE:

High-level class depends
on low-level class.

```
class EmailService {
    private SmtplibClient _client = new();
    public void Send(string msg) { ... }
}
```

GOOD EXAMPLE:

Depend on abstraction (interface).

```
public interface IEmailSender {
    void Send(string msg);
}
class EmailService {
    private readonly IEmailSender _sender;
    public EmailService(IEmailSender sender) {
        _sender = sender;
    }
    public void Send(string msg) {
        _sender.Send(msg);
    }
}
```

Why? Easy to test and extend.

WHY SOLID MATTERS?



✓ Scalable Codebase



✓ Easy to Maintain



✓ Easy to Test



✓ Fewer Bugs

✓ Better Collaboration

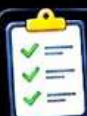


TAKEAWAY

Good code is not just
WORKING CODE.
It should be
EASY TO CHANGE.

“The cost of bad code
is paid every time
you change it.”

– Robert C. Martin



QUICK RECAP



One class
one job



Open for
extension



Behave like
base class



Small and
specific
interfaces



Depend on
abstractions

THINK ABOUT IT

If your code breaks when
you add a new feature...
is it really well designed?

DESIGN PATTERNS in .NET

PROVEN SOLUTIONS FOR REAL-WORLD PROBLEMS



REAL SCENARIO

You write code that works today.
But tomorrow it becomes:



Hard to extend



Hard to reuse



Hard to maintain



THE SOLUTION

Design patterns are proven, tested solutions to commonly occurring problems.



“Good developers write code.
Great developers use the right patterns.”



MOST USEFUL DESIGN PATTERNS IN .NET

1

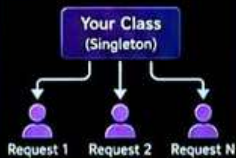
SINGLETON

You instance for the entire application.

WHEN TO USE?

- Logging
- Configuration
- Cache

HOW IT WORKS



EXAMPLE `Logger.cs`

```

public sealed class Logger
{
    private static readonly
    Logger _instance = new Logger();
    private Logger() {}

    public static Logger Instance
    {
        get { return _instance; }
    }

    public void Log(string message)
    {
        Console.WriteLine(message);
    }
}
    
```

BENEFIT

Ensures a single point of control.



2

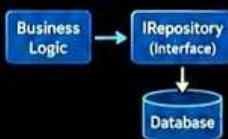
REPOSITORY

Separates data access logic from business logic.

WHEN TO USE?

- Database operations
- Clean Architecture
- Unit Testing

HOW IT WORKS



EXAMPLE `UserRepository.cs`

```

public interface IRepository
{
    Task<User> GetById(int id);
    Task<IEnumerable<User>> GetAll();
    Task Add(User user);
}

public class UserRepository :
    IRepository
{
    private readonly AppDbContext
    _context;
    public UserRepository(AppDbContext
    context)
    {
        _context = context;
    }

    public async Task<User> GetById(int id)
    {
        return await _context.Users
        .FindAsync(id);
    }
    // Other methods...
}
    
```

BENEFIT

Easy to test, maintain and switch data sources.



3

FACTORY

Creates objects without exposing instantiation logic.

WHEN TO USE?

- Multiple implementations
- Complex object creation
- Loose coupling

HOW IT WORKS



EXAMPLE `NotificationFactory.cs`

```

public interface INotification
{
    void Send(string message);
}

public class EmailNotification :
    INotification { ... }

public class SmsNotification :
    INotification { ... }

public class NotificationFactory
{
    public static INotification Create(
    string type)
    {
        return type.ToLower() switch
        {
            "email" => new EmailNotification(),
            "sms" => new SmsNotification(),
            _ => throw new ArgumentException(
            "Invalid type")
        };
    }
}
    
```

BENEFIT

Easy to add new types without changing client code.



4

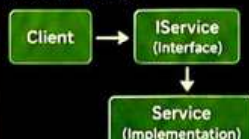
DEPENDENCY INJECTION

Inject dependencies instead of creating them.

WHEN TO USE?

- Loosely coupled systems
- Unit testing
- Scalable applications

HOW IT WORKS



EXAMPLE `Startup.cs / Program.cs`

```

// Register services
services.AddScoped<UserService,
    UserService>();

public class UserController :
    ControllerBase
{
    private readonly IUserService _service;
    public UserController(IUserService
    service)
    {
        _service = service;
    }
}
    
```

BENEFIT

Better testability, flexibility and maintainability.



MISTAKE I MADE

- Overusing patterns
- Using patterns where simple code was enough
- Result: Unnecessary complexity and harder code



WHEN TO USE PATTERNS?

- ✓ Repeated problems
- ✓ Complex or large systems
- ✓ Need for scalability
- ✓ Team collaboration
- ✓ When code becomes hard to change



REMEMBER

Patterns are tools, not goals.
Use wisely, not blindly.



TAKEAWAY

DON'T REINVENT SOLUTIONS.

USE PROVEN PATTERNS AND FOCUS ON BUILDING GREAT SOLUTIONS!

Think About It

If your code is getting messy, would you rewrite everything or apply a design pattern?



UP NEXT: Day 23 – Microservices vs Monolith
Which one to choose?



MICROSERVICES VS MONOLITH



WHICH ONE SHOULD YOU CHOOSE?



REAL SCENARIO

You're building a new application...
Would you go with:

A) Monolith or B) Microservices



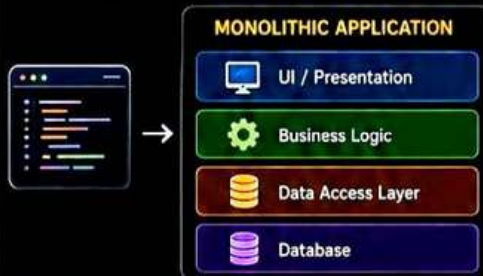
REAL UNDERSTANDING

Monolith = Simple, fast start

Microservices = Scalable, but complex

MONOLITH

Single application. Everything in one codebase.



✓ PROS

- Simple to build
- Easy to deploy
- Less operational overhead
- Easier testing (initially)

✗ CONS

- Hard to scale
- Changes affect entire application
- Technology lock-in
- Big codebase over time

MICROSERVICES

Application split into smaller, independent services.



✓ PROS

- Independent services
- Highly scalable
- Technology flexibility
- Better for large teams
- Fault isolation

✗ CONS

- Complex setup
- Hard to manage
- More network calls
- Requires DevOps maturity
- Higher operational cost



WHEN TO USE MONOLITH

- ✓ Small to medium applications
- ✓ Early-stage projects / MVP
- ✓ Small development team
- ✓ Simple business domain
- ✓ When speed is the priority



WHEN TO USE MICROSERVICES

- ✓ Large applications
- ✓ Multiple teams working
- ✓ Independent scaling required
- ✓ Complex business domains
- ✓ High availability & fault tolerance needed



MISTAKE I MADE

I thought microservices are always better.



RESULT

Over-engineering, unnecessary complexity and more maintenance.



TAKEAWAY

Don't follow trends.
Choose based on your **problem**, **team** and **goals**.

DECISION GUIDE (QUICK CHECK)

- | | |
|---|-----------------|
| ✓ Is your app simple or medium size? | → Monolith |
| ✓ Do you need independent scaling? | → Microservices |
| ✓ Do you have a large team? | → Microservices |
| ✓ Do you want faster delivery with less overhead? | → Monolith |



SCENARIO FOR YOU

If you're starting a new project today, would you directly choose microservices — or begin with a monolith?

A



MONOLITH

VS

B



MICROSERVICES



UP NEXT: Day 24 – API Versioning in .NET
How to evolve APIs safely

Day 23 | #SrinijaBuilds

CONSISTENCY
BUILDS EXPERTISE!

API VERSIONING IN .NET

How to evolve APIs without breaking users



REAL SCENARIO

You released an API...
Clients are using it in production.
Now you need to:

- Add new fields
- Change response structure

What happens if you change directly?

❗ Existing clients break! 🔥



WHAT IS API VERSIONING?

Managing changes in your API without breaking existing users.



Goal: Backward compatibility & smooth evolution

WHY IT MATTERS



Backward compatibility



Smooth upgrades



Better API lifecycle



Builds trust with users

COMMON APPROACHES

1

URL VERSIONING

Version is included in the URL path.

```
GET /api/v1/users
GET /api/v2/users
```



PROS

- Clear & explicit
- Easy to cache
- Widely used
- Best for public APIs

CONS

- URL becomes long
- More endpoints to maintain

EXAMPLE RESPONSE

```
GET /api/v1/users
```

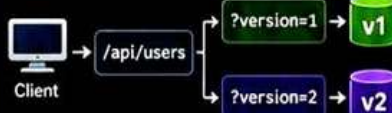
```
{
  "id": 1,
  "name": "Srinija",
  "email": "srinija@example.com"
}
```

2

QUERY PARAMETER

Version is passed as a query parameter.

```
GET /api/users?version=1
GET /api/users?version=2
```



PROS

- Keeps URL cleaner
- Easy to implement
- Flexible

CONS

- Can be missed by clients
- Not cache friendly

EXAMPLE RESPONSE

```
GET /api/users?version=2
```

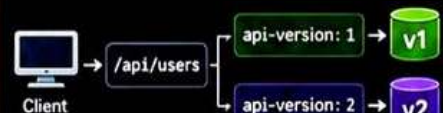
```
{
  "id": 1,
  "fullName": "Srinija R",
  "email": "srinija@example.com",
  "createdAt": "2024-05-20"
}
```

3

HEADER VERSIONING

Version is passed in the request header.

```
GET /api/users
Header: api-version: 1
Header: api-version: 2
```



PROS

- Keeps URL clean
- More flexible
- Good for internal APIs

CONS

- Harder to test manually
- Not visible in URL

EXAMPLE REQUEST

```
GET /api/users
```

Headers:

```
api-version: 2 {
  "id": 1,
  "fullName": "Srinija R",
  "email": "srinija@example.com",
  "createdAt": "2024-05-20"
}
```



BEST PRACTICES (What I learned)

- ✓ Use URL versioning for clarity.
- ✓ Keep old versions working.
- ✓ Deprecate gradually, not suddenly.
- ✓ Communicate version changes clearly.
- ✓ Document each version properly.



MISTAKE I MADE

Changing API without versioning.

RESULT:

- Broke frontend
- Production issues 😞
- Lost trust from users



WHEN TO VERSION YOUR API?

- 🟡 Breaking changes in request/response
- 🟡 Adding/removing required fields
- 🟡 Changing data types
- 🟡 Modifying business logic
- 🟡 Changing authentication/authorization
- 🟡 Major feature changes



TAKEAWAY

Never break existing clients.
Version your APIs properly and evolve them with care.



SCENARIO FOR YOU

If you need to update your API, would you break existing users — or version it properly?



VERSION IT PROPERLY



BREAK IT & UPDATE



UP NEXT: Day 25 – Authentication vs Authorization in .NET
Understanding the core difference!

TIP: Start with Monolith, Version your APIs, Scale with confidence! 🚀



AUTHENTICATION vs AUTHORIZATION



Don't confuse these!



REAL SCENARIO

A user tries to access your API...
What should happen first?

1 Authentication → 2 Authorization



KEY DIFFERENCE



Authentication = Identity
(Who are you?)



Authorization = Permission
(What can you access?)



WHY IT MATTERS

- Secure APIs
- Data protection
- Role-based access
- Better user experience

1 AUTHENTICATION (WHO ARE YOU?)

Verifies the identity of a user.

Examples:

- Login with username/password
- JWT token validation
- OAuth / External providers



Goal: Make sure the user is who they claim to be.

2 AUTHORIZATION (WHAT CAN YOU ACCESS?)

Checks what the authenticated user is allowed to do.

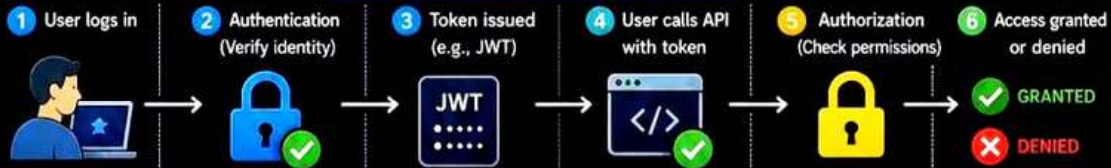
Examples:

- Admin can delete data
- User can only view data
- Access based on roles/claims



Goal: Make sure the user has permission to access the resource.

HOW IT WORKS – EXAMPLE FLOW



MISTAKE I MADE

I focused on authentication but skipped proper authorization.

Result?

- Users accessed restricted data
- Security issues
- Trust lost



REAL EXAMPLE

User	Role	Can Access
Admin	Admin	✓ All Endpoints
Manager	Manager	✓ Manage + View
User	User	✓ View Only
Guest	Guest	✓ Public Endpoints Only

COMMON IMPLEMENTATION IN .NET

Authentication

- [Authorize]
- JWT Bearer Token
- Identity / ASP.NET Core Identity



Authorization

- [Authorize(Roles = "Admin")]
- [Authorize(Policy = "CanEdit")]
- Claims / Policies / Roles



BEST PRACTICES



Always authenticate the user first.



Always authorize every request.



Follow Principle of Least Privilege.



Log and monitor access attempts.



Keep tokens secure & short-lived.

QUICK SUMMARY

Aspect	Authentication	Authorization
Question	Who are you?	What can you do?
Focus	Identity	Permissions
When	First step	After authentication
Example	Login / Token	Role / Policy check



TAKEAWAY

Authentication tells you WHO the user is.
Authorization tells you WHAT the user can do.

Do both. Do it right.



SCENARIO FOR YOU

If a user is logged in, should they be allowed to access everything – or should permissions still be checked?



UP NEXT: Day 26 – JWT Authentication in .NET
How tokens actually work!

Day 25 | #SrinijaBuilds

Security is not a feature,
it's a responsibility.





JWT AUTHENTICATION IN .NET



How tokens actually work



REAL SCENARIO

A user logs into your app...
Do you check credentials on EVERY request? ❌
I thought server validates user every time. But that's inefficient.



WHAT IS JWT?

- A token issued after login
- Used to verify user on every request
- Standard: JSON Web Token
- Compact, URL-safe and digitally signed



WHY JWT IS POWERFUL

- ✓ Stateless authentication
- ✓ No session storage on server
- ✓ Scalable APIs
- ✓ Cross-domain friendly
- ✓ Works great with microservices

HOW JWT WORKS (STEP BY STEP)



EXAMPLE (HEADER)

GET /api/users
Host: api.example.com
Authorization: Bearer <token>
Content-Type: application/json

VALIDATION ON SERVER

- ✓ Verify signature
- ✓ Check expiry
- ✓ Validate issuer/audience
- ✓ If valid → allow access
- ✓ If invalid → deny access



WHAT'S INSIDE JWT?

JWT has 3 parts (separated by dots)

HEADER . PAYLOAD . SIGNATURE

1 HEADER (Metadata)

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Algorithm used
Token type

2 PAYLOAD (Claims / Data)

```
{
  "sub": "123",
  "name": "Srinija R",
  "role": "Admin",
  "iat": 1715900000,
  "exp": 1715903600
}
```

User info
Claims
Issued at
Expiry time

3 SIGNATURE (Security)

HMACSHA256(
base64UrlEncode(header) + "." +
base64UrlEncode(payload), secretKey)

Ensures token
is not tampered

JWT EXAMPLE (DECODED)

```
{
  "sub": "123",
  "name": "Srinija R",
  "email": "srinija@example.com",
  "role": "Admin",
  "iat": 1715900000,
  "exp": 1715903600
}
```

Note: JWT is readable (Base64).
Do NOT store sensitive data.

❌ MISTAKE I MADE

I stored sensitive data in token.

Result?

- JWT is readable
- Anyone can decode it
- Security risk



BEST PRACTICES

- ✓ Keep token short
- ✓ Store only necessary claims
- ✓ Use strong secret keys
- ✓ Always validate signature & expiry
- ✓ Use HTTPS always
- ✓ Set proper expiry time
- ✓ Use refresh tokens for long sessions

TOKEN STORAGE (CLIENT SIDE)

Where to store JWT?



✓ HttpOnly Cookie (Recommended)
More secure against XSS

⚠️ LocalStorage
Easy but vulnerable to XSS

❌ sessionStorage
Better, but still accessible via JS

PROS & CONS OF JWT

✓ PROS

Stateless
Scalable
Cross-platform
No server memory
Fast

❌ CONS

Cannot be revoked
Token can be stolen
Payload is readable
Careful with expiry



QUICK SUMMARY

- ✓ Authenticate once (login)
- ✓ Get JWT token
- ✓ Send token in every request
- ✓ Server validates token
- ✓ Access granted or denied



SCENARIO FOR YOU

If your token expires,
would you force user
to login again —
or use refresh tokens?



TAKEAWAY

JWT makes authentication
efficient, scalable and secure
when implemented correctly.

Authenticate once,
use token everywhere!



UP NEXT: Day 27 – Refresh Tokens in .NET
How to handle token expiry!

Day 26 | #SrinijaBuilds

Security is not a feature,
it's a responsibility. 🔒



REFRESH TOKENS IN .NET

What happens when JWT expires?



REAL SCENARIO

Your JWT expires after 15 minutes...
What should happen?

- A) Force user to login again ❌
- B) Seamlessly renew session ✅

Earlier, I used to:

Force re-login

👉 Bad experience for users 😞



WHAT IS A REFRESH TOKEN?

A long-lived token used to generate new JWT access tokens

Access Token

Short-lived
(minutes)

Used in API calls

Refresh Token

Long-lived
(days/weeks)

Used to get new access tokens



WHY USE REFRESH TOKENS?

- ✅ Better user experience
- ✅ No frequent logins
- ✅ Secure session handling
- ✅ Works across devices



HOW REFRESH TOKENS WORK



FLOW EXAMPLE (REFRESH REQUEST)

```
// Access token expired
POST /refresh-token
Content-Type: application/json
{
  "refreshToken": "xyz123abc..."
}
```

SUCCESS RESPONSE

```
{
  "accessToken": "newAccessToken...",
  "expiresIn": 900
}
```

WHAT'S INSIDE JWT ACCESS TOKEN?

JWT has 3 parts separated by dot (.)

Header.Payload.Signature

HEADER

Contains info about the token type & signing algorithm

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD (CLAIMS)

Contains claims (user info, roles, permissions, etc.)

```
{
  "sub": "user123",
  "name": "Srinija",
  "role": "Admin",
  "exp": 1716200000
}
```

SIGNATURE

Created using header, payload and secret key

```
HMACHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload), secret)
```

KEY DIFFERENCE

Access Token

Short-lived
(5 - 30 min)
Sent in every API request
Stored in memory
(or secure storage)

Refresh Token

Long-lived
(days/weeks)
Used only to get new access token
Stored securely
(DB + HttpOnly cookie)



MISTAKE I MADE

Not storing refresh tokens securely

Result: Security risk 😞

DON'T DO THIS

```
localStorage.setItem(
  "refreshToken", token);
```



BEST PRACTICES

Store refresh tokens in DB (hashed)

- ✅ Use HttpOnly cookies (not JavaScript accessible)
- ✅ Rotate refresh tokens (issue new on each use)
- ✅ Invalidate old refresh tokens
- ✅ Invalidate on logout (clear from DB)
- ✅ Set proper expiration for refresh tokens
- ✅ Monitor & revoke suspicious tokens



SECURE STORAGE EXAMPLE

HTTPONLY COOKIE

```
Set-Cookie: refreshToken=xyz;
HttpOnly; Secure; SameSite=Strict;
Path=/; Max-Age=604800
```



DATABASE EXAMPLE

Id	UserId	Token (Hashed)	ExpiresAt
1	user123	a1b2c3d4e5f6...	2024-05-20



IMPORTANT NOTES

- ✅ Access token = Authentication
- ✅ Refresh token = Token renewal
- ✅ Never trust token blindly
- ✅ Always validate signature & expiry
- ✅ Revoke tokens when needed



TAKEAWAY

Don't force users to login repeatedly.
Use refresh tokens for smooth & secure experience.

Secure today, safe tomorrow! 🛡️



SCENARIO FOR YOU

If your token expires, would you log users out or refresh it seamlessly?



UP NEXT: Day 28 – Rate Limiting in .NET
How to protect your APIs!

Day 27 | #SrinijaBuilds

Security is not a feature,
it's a responsibility. 🛡️

RATE LIMITING IN .NET

How to protect your APIs from abuse & overload



PROTECT
PERFORM

REAL SCENARIO

What if someone sends 10,000 requests in 1 minute to your API?



- ✗ Server overload
- ✗ Performance issues
- ✗ Possible downtime

💡 That's where Rate Limiting comes in!

WHAT IS RATE LIMITING?

Restricting how many requests a client can make within a specific time period.



EXAMPLE IN .NET 8

Using built-in Rate Limiter Middleware

```
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("fixed", config =>
    {
        config.PermitLimit = 5; // Max 5 requests
        config.Window = TimeSpan.FromSeconds(10); // Per 10 sec
        config.QueueLimit = 0; // No queue
        config.QueueProcessingOrder =
            QueueProcessingOrder.OldestFirst;
    });
});
```

⚡ Only 5 requests allowed in every 10 seconds.

COMMON RATE LIMITING STRATEGIES

1 FIXED WINDOW

Allows a fixed number of requests in a fixed time window.



- ✓ Simple to implement
- ✗ Limit resets suddenly

2 SLIDING WINDOW

Tracks requests in a sliding time window for smoother control.



- ✓ More accurate
- ✗ Slightly complex

3 TOKEN BUCKET

Tokens are added over time. Each request consumes a token.



- ✓ Great for burst traffic
- ✗ Harder to configure

HOW RATE LIMITING WORKS (FLOW)



COMMON RESPONSE



Status: 429 Too Many Requests

Headers you might return:

- Retry-After: 10
- X-RateLimit-Limit: 5
- X-RateLimit-Remaining: 0
- X-RateLimit-Reset: 10

EXAMPLE RESPONSE

```
{
  "status": 429,
  "title": "Too Many Requests",
  "detail": "Rate limit exceeded.",
  "retryAfter": 10
}
```

💡 Always return proper 429 with helpful headers.

MISTAKE I MADE



Applying same limits to everyone.

Result:

- Legit users blocked
- Bad user experience
- Lost trust

BEST PRACTICES

- ✓ Different limits for different APIs
- ✓ Higher limits for trusted users
- ✓ Use IP + UserID for better control
- ✓ Return proper 429 response
- ✓ Log & monitor rate limit events
- ✓ Give clear message to clients
- ✓ Test limits and adjust over time

RATE LIMITING BY EXAMPLE

Client Type	Limit	Window	Example
Anonymous	10 req	1 min	Public APIs
Authenticated	100 req	1 min	Logged-in users
Premium User	1000 req	1 min	Paid users
Internal Service	10000 req	1 min	Trusted services

★ One size doesn't fit all!
Use smart limits.

TOOLS & LIBRARIES

- Built-in Rate Limiter (.NET 7+)
- AspNetCoreRateLimit
- Redis Rate Limiter
- Nginx / API Gateway Limits



TAKEAWAY

APIs need protection too!
Rate limiting improves security, stability & reliability.



SCENARIO FOR YOU

If someone floods your API with requests, how would you protect your server?



REMEMBER

- ✓ Protect your API
- ✓ Keep your server healthy
- ✓ Happy users, happy system



UP NEXT: Day 29 – CORS in .NET
Why browsers block your API requests!



CORS IN .NET

Why browsers block your API



REAL SCENARIO

Your frontend calls your API
But suddenly:

- ✗ Request blocked
- ✗ CORS error in browser
- ✗ Even though API works perfectly 😞



Earlier, I thought:
"Backend is broken"

But the real issue was:
CORS



WHAT IS CORS?

Cross-Origin Resource Sharing

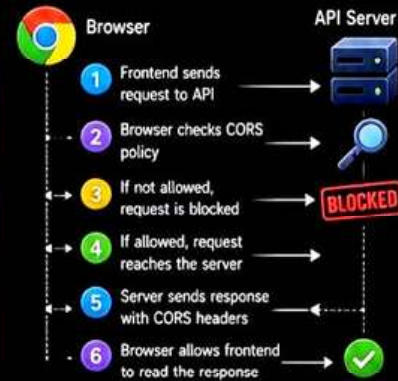
Browsers block requests when
frontend & backend come from
different origins.



Different ports / domains = Different origins

**Browser blocks request unless
backend allows it.**

BROWSER → API FLOW



EXAMPLE ORIGINS

Frontend:

`http://localhost:3000`

Backend:

`http://localhost:5000`

💡 Different ports = Different origins

ENABLE CORS IN .NET

1. Configure CORS Policy

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowFrontend",
        policy =>
        {
            policy.WithOrigins("http://localhost:3000")
                .AllowAnyHeader()
                .AllowAnyMethod();
        });
});
```

2. Use Middleware

```
app.UseCors("AllowFrontend");
```



Now browser will allow
requests from the allowed
origin.

COMMON CORS HEADERS

- Access-Control-Allow-Origin
- Access-Control-Allow-Methods
- Access-Control-Allow-Headers
- Access-Control-Allow-Credentials
- Access-Control-Max-Age

These headers tell the browser
what is allowed.

TYPES OF REQUESTS



Simple Request

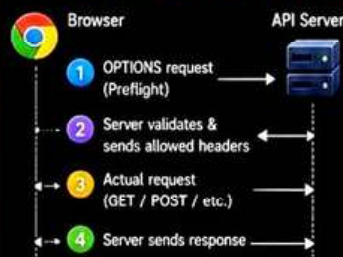
- GET, HEAD, POST
- Simple headers
- No preflight request



Preflight Request

- PUT, DELETE, PATCH, etc.
- Custom headers
- Browser sends OPTIONS request first to check permission

PREFLIGHT REQUEST FLOW (OPTIONS)



Example Preflight Request

```
OPTIONS /api/data HTTP/1.1
Origin: http://localhost:3000
Access-Control-Request-Method: POST
Access-Control-Request-Headers: content-type, authorization
```

⚠ MISTAKE I MADE

Using:

```
AllowAnyOrigin()
```

- ✗ Works quickly
- ✗ But risky in production 😞

DON'T DO THIS IN PRODUCTION

```
policy.AllowAnyOrigin()
.AllowAnyHeader()
.AllowAnyMethod();
```



BEST PRACTICES

- ✓ Allow specific origins
- ✓ Restrict methods if needed
- ✓ Allow only required headers
- ✓ Use credentials only if necessary
- ✓ Avoid overly open CORS policies
- ✓ Review CORS settings regularly



COMMON CORS ERROR IN BROWSER

Access to fetch at 'http://localhost:5000/api/data' from origin 'http://localhost:3000' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource. 😞



COMMON RESPONSE

HTTP

200 OK

(If allowed)

HTTP

429 Too Many Requests

(If rate limited)

CORS is about **ALLOWING**, Rate Limiting is about **LIMITING**.



WHY THIS MATTERS?

- ✓ Secure frontend-backend communication
- ✓ Prevent unauthorized browser requests
- ✓ Essential for modern web applications



TAKEAWAY

CORS is a browser security feature.
Your API must explicitly allow
trusted origins to communicate.

Allow smart. Not everything. 🔒



SCENARIO FOR YOU

If your frontend and backend run on
different domains, how would you
securely allow communication?



UP NEXT: Day 30 – Middleware in .NET
How requests flow through your app.

Day 29 | #SrinijaBuilds

Security is not a feature,
it's a responsibility. 🔒



MIDDLEWARE IN .NET

How requests flow through your app



REAL SCENARIO

A request enters your API...
What happens before it reaches your controller?

Earlier, I thought:

Request directly hits controller 😊

But actually:

It passes through multiple layers first.

That's called:

Middleware Pipeline!



WHAT IS MIDDLEWARE?

Small components in the request pipeline.
Each middleware handles something specific.

Examples:

- Authentication
- Authorization
- Logging
- Exception handling
- CORS
- Rate limiting
- And more...

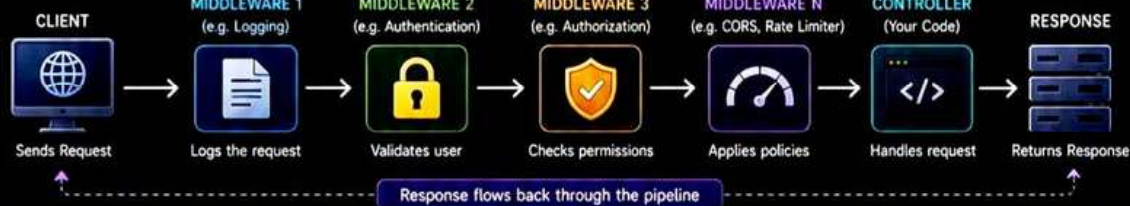


WHY MIDDLEWARE MATTERS?

- Centralized processing
- Cleaner architecture
- Better request handling
- Reusability across the app
- Each piece has a single responsibility



REQUEST FLOW (MIDDLEWARE PIPELINE)



EXAMPLE IN .NET

```

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

// 1. Exception handling
app.UseExceptionHandler("/error");

// 2. HTTPS Redirection
app.UseHttpsRedirection();

// 3. CORS
app.UseCors("AllowFrontend");

// 4. Authentication
app.UseAuthentication();

// 5. Authorization
app.UseAuthorization();

// 6. Rate limiting
app.UseRateLimiter();

// 7. Endpoints
app.MapControllers();

app.Run();
    
```



ORDER MATTERS (EXAMPLE)



WRONG ORDER

UseAuthorization → UseAuthentication → Controller

What happens?

- Authorization runs before authentication.
- User is not authenticated, so access is denied.



CORRECT ORDER

UseAuthentication → UseAuthorization → Controller

Result:

- User is authenticated first.
 - Then authorization checks permissions.
- Everything works as expected! ✓

WHAT CAN MIDDLEWARE DO?



Process the request

Perform actions before passing it to the next middleware.



Modify the request

Change headers, body, or other details.



Stop the request

Return a response early (e.g., 401, 403, 429).



Process the response

Modify response before sending it back to client.

COMMON MIDDLEWARE

- UseExceptionHandler
Handles errors gracefully
- UseHttpsRedirection
Redirects HTTP to HTTPS
- UseCors
Handles Cross-Origin Resource Sharing
- UseAuthentication
Validates user identity
- UseAuthorization
Checks user permissions
- UseRateLimiter
Protects API from abuse

MISTAKE I MADE

Using middleware in wrong order.

Result:

- Authentication failed
- CORS issues
- Unexpected behavior



```

app.UseAuthorization();
app.UseAuthentication();
// Wrong Order!
    
```

BEST PRACTICES

- Always use correct order of middleware.
- Keep middleware small and focused.
- Use built-in middleware when possible.
- Handle exceptions globally.
- Log important events.
- Test your pipeline thoroughly.



VISUAL SUMMARY



KEY TAKEAWAY

Every request travels through the middleware pipeline.

Middleware order can make or break your API.

Understand the flow, control the pipeline, build better APIs! 🚀



SCENARIO FOR YOU

If authentication middleware runs AFTER authorization... what do you think happens? 😊



UP NEXT

Day 31 - Dependency Injection in .NET
How services are managed internally.



30 DAYS COMPLETED!

Keep building. Keep learning.
You're doing amazing! 🔥



DEPENDENCY INJECTION IN .NET

How .NET manages services internally



REAL SCENARIO

Your controller needs:

- Logger
- Repository
- Email service
- And more...

Should you create them manually? ❌

```
var repo = new UserRepository();
var logger = new Logger();
var email = new EmailService();
```

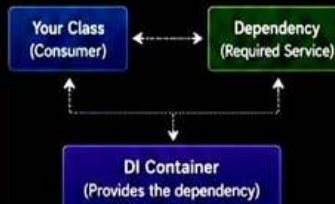
Problems:

- ❌ Tight coupling
- ❌ Hard to test
- ❌ Hard to maintain



WHAT IS DEPENDENCY INJECTION?

Instead of creating dependencies manually, .NET provides them automatically.



You ask for a service, DI container gives it to you.



EXAMPLE

```
public class UserService
{
    private readonly IUserRepository _repo;

    // Dependency comes from DI container
    public UserService(IUserRepository repo)
    {
        _repo = repo;
    }

    public void GetUser()
    {
        _repo.GetUser();
    }
}
```



No manual creation.
Dependency comes from DI container.



REGISTER SERVICES

Tell .NET how to create and provide services.

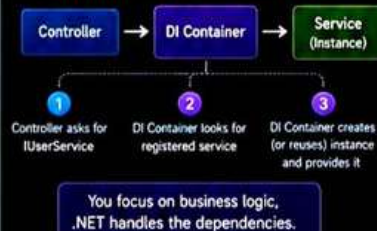
In Program.cs / Startup:

```
builder.Services.AddScoped<IUserRepository, UserRepository>();
builder.Services.AddSingleton<IEmailService, EmailService>();
builder.Services.AddTransient<ILoggerService, LoggerService>();
```

Interface → Implementation

Lifetime → How long the instance lives

HOW IT WORKS (BEHIND THE SCENES)



WHY DI MATTERS?

- ✓ Loose coupling
- ✓ Easier unit testing
- ✓ Better maintainability
- ✓ Cleaner code
- ✓ Follows SOLID principles
- ✓ Built-in by the framework

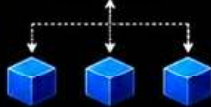


Better architecture
Better applications

SERVICE LIFETIMES (VERY IMPORTANT!)

1 TRANSIENT

New instance every time it is requested.



Use for lightweight, stateless services.

2 SCOPED

One instance per request (same instance in that request).



Default for most services in ASP.NET Core.

3 SINGLETON

One instance for entire application lifetime.



Use for stateless, thread-safe services.



WHEN TO USE WHAT?

Lifetime	Use When
Transient	When you need a new instance every time.
Scoped	For services tied to a user request (repositories, DbContext).
Singleton	For shared, stateless services (缓存, config, logging).



MISTAKE I MADE

Using Singleton for everything 😊

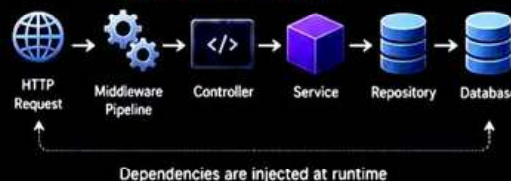
Result:

- ❌ Shared state issues
- ❌ Unexpected bugs
- ❌ Hard to debug

```
builder.Services.AddSingleton<IUserRepository, UserRepository>(); // Wrong!
```

Always choose the right lifetime. 😞

DI FLOW IN A REAL APP



KEY TAKEAWAY

- ✓ Don't create dependencies manually.
- ✓ Let .NET DI Container manage them.
- ✓ Choose the right lifetime.
- ✓ Loosely coupled code is easier to test and maintain.



DI is not just syntax, it's an architecture pattern.



BEST PRACTICES

- ✓ Depend on abstractions (interfaces)
- ✓ Register services in one place
- ✓ Use the right lifetime
- ✓ Keep services small and focused
- ✓ Avoid Service Locator pattern



SCENARIO FOR YOU

If your controller creates all dependencies manually, how easy would testing become? What problems would you face? 😞



UP NEXT

Day 32 – Repository Pattern in .NET
Separating data access cleanly.



YOU'RE BUILDING SOMETHING GREAT!

Keep learning. Keep building.

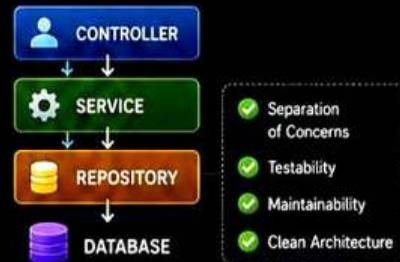
#SrinijaBuilds

31



REPOSITORY PATTERN IN .NET

Separating data access cleanly



REAL SCENARIO

Your controller directly writes SQL queries or accesses DB...

Works initially, but later:

- ✗ Hard to maintain
- ✗ Hard to test
- ✗ Business logic + DB logic mixed together

EXAMPLE (Not Recommended)

```

public IActionResult GetUsers()
{
    var data = _context.Users.ToList();
    return Ok(data);
}
  
```



WHAT IS REPOSITORY PATTERN?

A layer between business logic and database access. It hides database implementation details and provides a clean API.



The business logic talks to the repository, not directly to the database.



WHY REPOSITORY PATTERN?

- ✓ Cleaner architecture
- ✓ Easier testing (mocking)
- ✓ Better separation of concerns
- ✓ Reusable data access logic
- ✓ Easier to change database without affecting business logic



EXAMPLE IMPLEMENTATION

1 INTERFACE (CONTRACT)

```

public interface IUserRepository
{
    List<User> GetUsers();
    User GetUserById(int id);
    void AddUser(User user);
    void UpdateUser(User user);
    void DeleteUser(int id);
}
  
```

2 REPOSITORY IMPLEMENTATION

```

public class UserRepository : IUserRepository
{
    private readonly AppDbContext _context;

    public UserRepository(AppDbContext context)
    {
        _context = context;
    }

    public List<User> GetUsers()
    {
        return _context.Users.ToList();
    }

    // Other methods...
}
  
```

3 REGISTER IN DI CONTAINER

```

// In Program.cs or Startup.cs
builder.Services.AddScoped<IUserRepository,
    UserRepository>();
  
```



Now .NET DI will provide UserRepository wherever IUserRepository is needed.

REAL FLOW IN APPLICATION



RESPONSIBILITY SEPARATION

REPOSITORY HANDLES

- ✓ Data access
- ✓ Queries
- ✓ CRUD operations
- ✓ Database related logic

SERVICE HANDLES

- ✓ Business logic
- ✓ Validation
- ✓ Rules
- ✓ Orchestration



MISTAKE I MADE

Putting ALL logic inside repositories 😞

Result:

- ✗ Fat repositories
- ✗ Hard to maintain
- ✗ Hard to test

```

public class UserRepository : IUserRepository
{
    // 500+ lines of business logic
    // Validations, calculations,
    // external calls, etc.
}
  
```

💡 Keep it clean and focused!



WHEN TO USE?

- ✓ When your application has multiple data sources
- ✓ When you want to write unit tests easily
- ✓ When you want to follow clean architecture
- ✓ When data access logic is complex or shared



BENEFITS IN REAL WORLD



KEY TAKEAWAY

- ➡ Don't access the database from controllers or services directly.
- ➡ Use repositories to abstract data access.
- ➡ Keep repositories focused on data access, not business logic.
- ➡ Clean code = Scalable code!



A small pattern that makes a BIG difference in large applications.



TESTING MADE EASY

You can mock the repository easily!

```

// Example with Moq
var mockRepo = new Mock<IUserRepository>();
mockRepo.Setup(r => r.GetUsers())
    .Returns(new List<User>() { new User() });

var service = new UserService(mockRepo.Object);
var result = service.GetUsers();

// Easy to test! 🟢
  
```



IN SHORT

- ✓ Repository Pattern = Cleaner architecture
- ✓ It separates concerns and makes your code testable and maintainable
- ✓ A must-have pattern for enterprise applications



UP NEXT

Day 33 – Unit of Work Pattern
Managing transactions properly across multiple operations.

DAY 33

OF 100 DAYS .NET BACKEND JOURNEY



UNIT OF WORK PATTERN IN .NET

Managing transactions properly



REAL SCENARIO

Imagine this flow:

- 1 Create Order
- 2 Update Inventory
- 3 Save Payment

What if Payment fails AFTER order is saved?

Now your database becomes **inconsistent**.

WHAT I USED TO DO (WRONG WAY) 😞

```
_context.Orders.Add(order);
_context.SaveChanges();

_context.Payments.Add(payment);
_context.SaveChanges();
_context.Inventory.Update(stock);
_context.SaveChanges();
```

❌ Dangerous in real applications

WHAT IS UNIT OF WORK?



A pattern that groups multiple database operations into ONE transaction.

So either:

- ✅ Everything succeeds
- ❌ Everything rolls back



It ensures data consistency.

★ WHY IT MATTERS?

- ✅ Prevents partial updates
- ✅ Maintains data consistency
- ✅ Better transaction handling
- ✅ Cleaner architecture
- ✅ Easier error handling

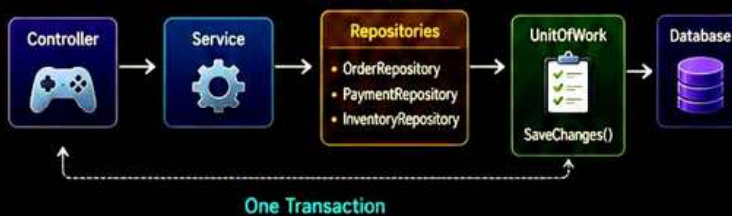
THE RIGHT WAY (WITH UOW) ✅

```
_context.Orders.Add(order);
_context.Payments.Add(payment);
_context.Inventory.Update(stock);

await _unitOfWork.SaveChangesAsync();
```

All or nothing!

HOW IT WORKS



REPOSITORY vs UNIT OF WORK



EXAMPLE SCENARIO



Keeps your system consistent!

SAMPLE INTERFACE & IMPLEMENTATION

IUnitOfWork

```
public interface IUnitOfWork
{
    Task<int> SaveChangesAsync();
}
```



UnitOfWork Implementation

```
public class UnitOfWork : IUnitOfWork
{
    private readonly ApplicationDbContext _context;
    public UnitOfWork(ApplicationDbContext context)
    {
        _context = context;
    }
    public async Task<int> SaveChangesAsync()
    {
        return await _context.SaveChangesAsync();
    }
}
```

⚠️ MISTAKE I MADE

Calling SaveChanges everywhere 😞

Result:

- ❌ Inconsistent data
- ❌ Hard-to-track bugs
- ❌ Poor performance



⌚ WHEN TO USE?

- ✅ When multiple steps must succeed or fail as one
- ✅ When your application has complex transactions
- ✅ When data consistency is critical
- ✅ In financial, banking, booking, order systems

🏆 BENEFITS IN REAL WORLD

- 🛡️ Data Consistency
- 📊 Better Reliability
- 🐛 Fewer Bugs
- 🚀 Scalable Architecture

📖 IMPORTANT NOTES

- Unit of Work does not handle business logic.
- It only manages transactions.
- Combine with Repository Pattern for clean architecture.

🎯 IN SHORT

- Multiple DB operations should behave as ONE unit.
- Unit of Work ensures ACID principles.
- Clean, reliable and professional way to handle transactions.



🚀 UP NEXT

Day 34 - CQRS Pattern in .NET
Separating Reads & Writes
for better performance
and scalability.



You're building something great!

Keep learning. Keep building.

#SrinijaBuilds

33



CQRS PATTERN IN .NET

Separating Reads & Writes cleanly

PROBLEM

Reads and Writes are handled together.



SOLUTION

Separate them for better scalability.



REAL SCENARIO

- Your application:
- Reads huge amounts of data
 - Handles complex updates

Both are handled using the SAME models and logic.

Result?

- ✗ Complex code
- ✗ Poor scalability
- ✗ Hard maintenance



WHAT IS CQRS?

CQRS (Command Query Responsibility Segregation) separates READ operations and WRITE operations into different models and handlers.



QUERY
(Read)

Reads data
Does NOT change data
Example:
GetUsersQuery

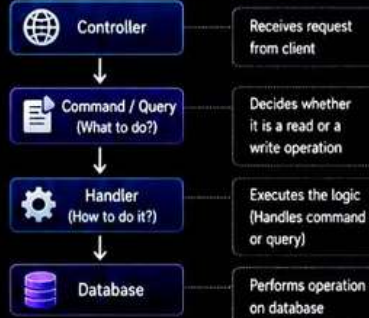


COMMAND
(Write)

Changes data
Insert / Update / Delete
Example:
CreateUserCommand

Different models for different responsibilities.

HOW CQRS WORKS (FLOW)



BEFORE CQRS (THE OLD WAY) 😞

```

public class UserService
{
    public List<User> GetUsers() { ... }
    public void AddUser(User user) { ... }
    public void UpdateUser(User user) { ... }
    public void DeleteUser(int id) { ... }
}
  
```

One service does everything.
Hard to maintain and scale.

AFTER CQRS (THE NEW WAY) 😊

QUERY SIDE (Read Model)

```

GetUsersQuery
GetUserByIdQuery
GetUserStatsQuery
...
  
```

Optimized for reads.
Can use different DB.

COMMAND SIDE (Write Model)

```

CreateUserCommand
UpdateUserCommand
DeleteUserCommand
...
  
```

Optimized for writes.
Ensures data consistency.

Each side has its own model, handler and optimization.

EXAMPLE: COMMAND

```

public record CreateUserCommand(string Name);
public class CreateUserHandler
{
    private readonly IUserRepository _repo;
    public CreateUserHandler(IUserRepository repo)
    {
        _repo = repo;
    }
    public async Task Handle(CreateUserCommand cmd)
    {
        var user = new User { Name = cmd.Name };
        await _repo.AddAsync(user);
    }
}
  
```

★ WHY CQRS MATTERS?

- ✓ Cleaner separation of concerns
Read and write have different needs.
- ✓ Better scalability
Scale reads and writes independently.
- ✓ Easier maintenance
Changes in one side won't affect the other.
- ✓ Independent optimization
Use different databases, caching, models.

👍 WHEN TO USE CQRS?

- ✓ Large applications with complex domains
- ✓ High read/write traffic
- ✓ Different performance requirements for reads and writes
- ✓ When your team can handle the added complexity
- ✓ When you need independent scaling for read and write operations

👎 WHEN NOT TO USE CQRS?

- ✗ Small or medium CRUD applications
- ✗ Simple business logic
- ✗ Low traffic applications
- ✗ When the team is not familiar with this pattern

⚠ Over-engineering is also a problem!

CQRS ARCHITECTURE OVERVIEW



🔑 KEY TAKEAWAY

- ✓ CQRS separates reads and writes.
- ✓ Different models, different handlers.
- ✓ Better scalability and maintainability.
- ✓ Use it when your application is complex and growing.

★ Reads and writes think differently.
So design them differently.

⚠ MISTAKE I MADE

Tried CQRS in small projects 😞

Result:

- ✗ Over-engineering
- ✗ Increased complexity
- ✗ Not worth the effort

Use the right tool for the right problem.



🏠 REAL WORLD EXAMPLE

In a banking application:

- Queries: Check balance, transaction history (high read)
- Commands: Transfer money, update details (high write, needs consistency)

CQRS helps handle both efficiently.



UP NEXT

Day 35 – Mediatr in .NET

Simplifying CQRS implementation and in-process messaging.



DAY 35



MEDIATR IN .NET

Making CQRS Cleaner & Scalable



REAL SCENARIO

Your controller starts looking like this:

```
public class UserController : ControllerBase
{
    private readonly IUserService _service;
    private readonly ILogger _logger;
    private readonly IEmailService _email;
    private readonly ICacheService _cache;
}
```

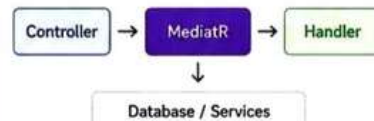


Too many dependencies... Hard to manage!



WHAT IS MEDIATR?

Mediatr is a library that implements the Mediator Pattern.



BEFORE MEDIATR

Controllers calling services directly.

Works initially...

But over time:

- ✗ Tight coupling
- ✗ Fat controllers
- ✗ Hard maintenance
- ✗ Hard to test



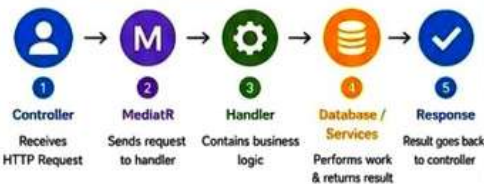
AFTER MEDIATR

Controller → Mediatr → Handler

- ✓ Clean & thin controllers
- ✓ Better separation of concerns
- ✓ Easy CQRS implementation
- ✓ Scalable architecture



HOW IT WORKS (FLOW)



Decouples sender and receiver. Promotes clean, maintainable code.

EXAMPLE: SIMPLE COMMAND & HANDLER

1. COMMAND

```
public record CreateUserCommand
(string Name) : IRequest<int>;
```

`IRequest<int>` means this command returns an int

3. CONTROLLER

```
[ApiController]
[Route("api/[controller]")]
public class UsersController : ControllerBase
{
    private readonly IMediator _mediator;

    public UsersController(IMediator mediator) => _mediator = mediator;

    [HttpPost]
    public async Task<ActionResult> CreateUser(
        [FromBody] CreateUserCommand command)
    {
        var id = await _mediator.Send(command);
        return Ok(new { Id = id });
    }
}
```

2. HANDLER

```
public class CreateUserHandler
: IRequestHandler<CreateUserCommand, int>
{
    private readonly IUserRepository _repo;

    public CreateUserHandler(IUserRepository repo) => _repo = repo;

    public async Task<int> Handle(
        CreateUserCommand request,
        CancellationToken token)
    {
        var user = new User { Name = request.Name };
        _repo.Add(user);
        await _repo.SaveChangesAsync(token);
        return user.Id;
    }
}
```

Controller only knows about IMediator, not the handler or services!

COMMON USE CASES

- CQRS**
Separate read (Query) and write (Command) operations
- Notifications**
Send notifications between parts of the application
- Domain Events**
Publish domain events inside the application
- Clean Architecture**
Keep controllers thin & business logic in handlers

MISTAKE I MADE

Using Mediatr for very small projects

Result:

- ✗ Extra abstraction
- ✗ Unnecessary complexity



REAL UNDERSTANDING

Mediatr is NOT magic. It's a way to organize communication cleanly in your app.

TOOLS & LIBRARIES



Built-in Rate Limiter (NET 7+)



ASP.NET Core RateLimit



Redis Rate Limiter



Nginx / API Gateway Limits

HOW TO REGISTER MEDIATR

```
services.AddMediatr(cfg =>
    cfg.RegisterServicesFromAssembly(
        typeof(Program).Assembly));
```



Registers all requests, handlers & notifications from the assembly.

RATE LIMITING BY EXAMPLE

Client Type	Limit	Window	Example
Anonymous	10 req	1 min	Public APIs
Authenticated	100 req	1 min	Logged-in users
Premium User	1000 req	1 min	Paid users
Internal Service	10000 req	1 min	Trusted services

★ One size doesn't fit all! Use smart limits.

BEST PRACTICES

- ✓ Keep handlers small and focused
- ✓ One request / command = One handler
- ✓ Use meaningful names for requests
- ✓ Return only what the client needs
- ✓ Use CancellationToken
- ✓ Log only important events

WHEN TO USE MEDIATR?

- ✓ Medium to large applications
- ✓ When using CQRS
- ✓ Complex business logic
- ✓ When you need better separation of concerns



WHEN NOT TO USE?

- ✗ Very small CRUD applications
- ✗ Simple project with minimal logic
- ✗ When your team is not familiar with the pattern



TAKEAWAY

Controllers should stay thin. Business logic belongs in handlers.

Mediatr helps you build clean, maintainable and scalable .NET applications.



SCENARIO FOR YOU

If your controller has too many dependencies, is your architecture really clean?



UP NEXT

Day 36 – DTO vs Entity in .NET
Why separating them matters



#dotnet

#csharp

#backenddevelopment

#webapi

#softwareengineering

#100DaysOfCode

#LearningInPublic

#BackendJourney

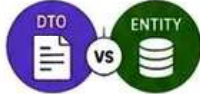
#SrinijaBuilds



DAY 36

DTO vs ENTITY in .NET

Why separating them matters



BEFORE: I used to do this

Return entities directly from API

```
public IActionResult GetUser(int id)
{
    var user = _db.Users.Find(id);
    return Ok(user); // returns entity
}
```

Works initially...

But later:

- ✗ Exposes sensitive data
- ✗ Tight coupling with DB
- ✗ Hard API versioning
- ✗ Breaks frontend unexpectedly



AFTER: I learned DTOs

Use DTOs (Data Transfer Objects) to shape data for the client

```
public IActionResult GetUser(int id)
{
    var user = _db.Users.Find(id);
    var dto = new UserDto
    {
        Email = user.Email
    };
    return Ok(dto); // return DTO
}
```

- ✓ Hides sensitive data
- ✓ Better API contracts
- ✓ Easier versioning
- ✓ Cleaner architecture



REAL SCENARIO

Your database entity looks like this:

```
public class User
{
    public int Id { get; set; }
    public string Email { get; set; }
    public string PasswordHash { get; set; }
    public bool IsAdmin { get; set; }
}
```

⚠ Now imagine returning this directly from your API 😬

WHAT'S THE DIFFERENCE?

ENTITY

- Represents database table / model
- Used internally in application & DB layer

Example:

```
User Product Order
...
```

DTO

- Object used ONLY for transferring data
- Between API and client

Example:

```
public class UserDto
{
    public string Email { get; set; }
}
```

HOW IT WORKS (FLOW)



Entity stays inside. DTO goes outside.

EXAMPLE

1. ENTITY (Database Model)

```
public class User
{
    public int Id { get; set; }
    public string Email { get; set; }
    public string PasswordHash { get; set; }
    public bool IsAdmin { get; set; }
    public DateTime CreatedAt { get; set; }
}
```

2. DTO (Data Transfer Object)

```
public class UserDto
{
    public int Id { get; set; }
    public string Email { get; set; }
    // No PasswordHash
    // No IsAdmin
    // No CreatedAt
}
```

📌 DTO contains only the data that should be exposed to the client.

WHY DTOS MATTER

- 🛡 **Hide Sensitive Fields**
Don't expose data like PasswordHash, IsAdmin, internal IDs, etc.
- 📄 **Better API Contracts**
You control exactly what the client gets. Stable and predictable.
- 📦 **Easier Versioning**
Change internal models without breaking APIs.
- 👥 **Cleaner Architecture**
Separation of concerns. Better maintainability & scalability.

✗ MISTAKE I MADE

Using entities everywhere 😬

Result:

- ✗ Sensitive data leaks
- ✗ Breaking frontend changes
- ✗ Poor maintainability



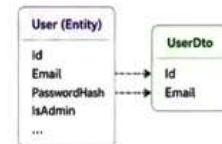
REAL UNDERSTANDING

Entity = Internal model
Used within the system
DTO = External contract
Used for communication with outside world



COMMON MAPPING (MANUAL)

```
var dto = new UserDto
{
    Id = user.Id,
    Email = user.Email
};
```



★ BONUS

This is why tools like AutoMapper exist ++



WHEN TO USE DTOS?

- ✓ Public APIs (Web APIs)
- ✓ Microservices communication
- ✓ External integrations
- ✓ Mobile / SPA applications
- ✓ Different data shape required

WHEN NOT TO USE DTOS?

- ✗ Very simple CRUD internal applications
- ✗ No external exposure
- ✗ Tiny projects (if not needed)
- ✗ If entity shape is already perfect (rare case 😊)



BEST PRACTICES



- ✓ Always validate & sanitize data
- ✓ Keep DTOs in a separate project / folder
- ✓ Use AutoMapper for larger projects
- ✓ Don't expose entities directly
- ✓ Keep DTOs small and focused

🏆 KEY TAKEAWAY

Don't expose database entities directly.
Use DTOs to control your API responses cleanly and securely.

Good APIs are about good contracts!

💬 SCENARIO FOR YOU

If your database structure changes, should your API responses break too?



UP NEXT

Day 37 - AutoMapper in .NET
Simplifying DTO mapping



#dotnet

#csharp

#backenddevelopment

#webapi

#softwareengineering

#100DaysOfCode

#LearningInPublic

#BackendJourney

#SrinijaBuilds



DAY 37



AutoMapper in .NET

Simplifying DTO Mapping

THE REAL SCENARIO

You have multiple DTOs and entities...

And your code starts looking like this everywhere 🤔

```
var dto = new UserDto
{
    Id = user.Id,
    Email = user.Email,
    Name = user.Name
};
```

Works initially...

But later:

- ✗ Repetitive code
- ✗ Hard maintenance
- ✗ Mapping everywhere



SOLUTION: AUTOMAPPER

A library that automatically maps:

- ✓ Entity → DTO
- ✓ DTO → Entity

With minimal manual code.



AutoMapper

★ WHY AUTOMAPPER MATTERS

- Less repetitive code
- Cleaner services / controllers
- Easier maintenance
- Centralized mapping logic

EXAMPLE: SIMPLE MAPPING

1. ENTITY

```
public class User
{
    public int Id { get; set; }
    public string Email { get; set; }
    public string Name { get; set; }
}
```



2. DTO

```
public class UserDto
{
    public int Id { get; set; }
    public string Email { get; set; }
    public string Name { get; set; }
}
```



3. MAPPING PROFILE

```
public class MappingProfile : Profile
{
    public MappingProfile()
    {
        CreateMap<User, UserDto>();
    }
}
```



4. USAGE

```
var dto = _mapper.Map<UserDto>(user);
```



User (Entity)



AutoMapper



UserDto (DTO)

HOW IT WORKS (FLOW)



AutoMapper sits between your entity and DTO and handles the mapping for you.

✗ MISTAKE I MADE

Using AutoMapper for EVERYTHING 🤔

Result:

- ✗ Hard debugging
- ✗ Hidden mapping issues
- ✗ Overcomplicated profiles



REAL UNDERSTANDING

AutoMapper is helpful

BUT

Manual mapping is sometimes clearer.

Especially for:

- ✓ Complex transformations
- ✓ Performance-critical logic



BEST PRACTICE

Use AutoMapper for:

- ✓ Simple mappings (1:1 properties, straightforward cases)

Avoid overusing it for:

- ✗ Complex business logic (use manual mapping)



Always keep mapping profiles small, clean and focused.

COMMON SCENARIOS

- Entity → DTO
Returning data in API responses
- DTO → Entity
Accepting input from client
- List Mapping
`IEnumerable<User> → List<UserDto>`
- Nested Objects
Mapping complex / nested properties
- Ignoring Members
Ignore unwanted properties easily

TOOLS & INTEGRATION



NuGet Package
AutoMapper



Works with
ASP.NET Core



Easy to register
in DI container

HOW TO REGISTER AUTOMAPPER

```
services.AddAutoMapper(typeof(MappingProfile).Assembly);
# or
services.AddAutoMapper(AppDomain.CurrentDomain.GetAssemblies());
```



Registers all
profiles and
enables
mapping.

WHEN TO USE?

- ✓ Medium to large applications
- ✓ When mapping is repetitive
- ✓ When using DTOs
- ✓ When you want cleaner code

WHEN NOT TO USE?

- ✗ Very simple CRUD apps with few fields
- ✗ Complex business transformations
- ✗ Performance critical hotspots
- ✗ When team is not familiar with the tool



KEY TAKEAWAY

AutoMapper reduces boilerplate mapping code. But simplicity and readability still matter most.

Good code is about clarity, not just less code.



SCENARIO FOR YOU

If mapping logic is repeated everywhere, how maintainable will your code become over time?



UP NEXT

Day 38 - Clean Architecture in .NET
Structuring scalable applications



#dotnet

#csharp

#backenddevelopment

#webapi

#softwareengineering

#100DaysOfCode

#LearningInPublic

#BackendJourney

#SrinijaBuilds



DAY 38



CLEAN ARCHITECTURE

in .NET

Structuring scalable applications

THE REAL SCENARIO

As your project grows...

- ✗ Business logic everywhere
- ✗ Controllers become huge
- ✗ Database tightly coupled
- ✗ Hard testing & maintenance

EARLIER: I used to do this

Everything in one project

- Controllers
- Services
- Database
- Business Logic
- All mixed together



! Works for small apps... But becomes messy over time.

WHAT IS CLEAN ARCHITECTURE?

A way to structure applications by separating responsibilities into layers.

- ✓ Maintainability
- ✓ Scalability
- ✓ Testability

“ The goal is to keep your business rules independent of frameworks and external concerns.



THE LAYERS (INSIDE → OUTSIDE)

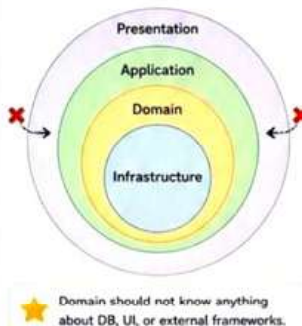


LAYER RESPONSIBILITIES (DETAILS)



IMPORTANT RULE

Inner layers should NOT depend on outer layers.



MISTAKE I MADE

Trying to apply Clean Architecture too early

- Result:
- ✗ Over-engineering
 - ✗ Too many projects
 - ✗ Unnecessary complexity



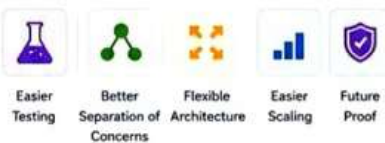
REAL UNDERSTANDING

Clean Architecture is useful when:

- ✓ Applications become large
- ✓ Teams grow
- ✓ Maintainability matters

Not every small app needs it.

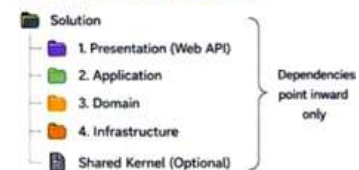
WHY COMPANIES USE CLEAN ARCHITECTURE



HOW IT WORKS (FLOW)



TYPICAL PROJECT STRUCTURE



DO'S

- ✓ Keep business logic in Domain layer
- ✓ Depend on abstractions, not concretions
- ✓ Use Dependency Injection
- ✓ Write unit tests for core logic
- ✓ Keep layers independent

DON'TS

- ✗ Don't put business logic in Controllers
- ✗ Don't access DB directly from Application
- ✗ Don't create circular dependencies
- ✗ Don't expose internal models to API
- ✗ Don't over-engineer for small projects

COMMON PATTERNS USED

- Repository Pattern
- Dependency Injection
- CQRS Pattern
- Unit of Work
- Domain-Driven Design (DDD)

TOOLS & TECHNOLOGIES (EXAMPLE)

- ASP.NET Core (API)
- Entity Framework Core
- MediatR (CQRS)
- AutoMapper (Mapping)
- xUnit / NUnit (Testing)

KEY TAKEAWAY

Clean Architecture separates responsibilities clearly. Good architecture makes large applications manageable.

Invest in architecture. It pays off later!

SCENARIO FOR YOU

If your database changes tomorrow, should your entire application break? Think about it!



UP NEXT

Day 39 – Onion Architecture in .NET
How it differs from Clean Architecture



REMEMBER

Good code works. Good architecture keeps it working for years.

DAY 39



ONION ARCHITECTURE in .NET

How it differs from Clean Architecture

THE REAL SCENARIO

Your application grows bigger...

You now want:

- ✓ Flexible architecture
- ✓ Independent business logic
- ✓ Easy testing
- ✓ Minimal coupling

But how should dependencies flow? 🤔

EARLIER, I THOUGHT:

Onion Architecture and Clean Architecture are exactly the same 😊

But they're similar...
NOT identical.



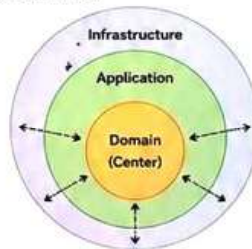
WHAT IS ONION ARCHITECTURE?

An architecture pattern where:

- Core business logic stays at the center
- Outer layers depend on inner layers

Think of it like:

🍷 Layers of an onion



Dependencies point inward.

TYPICAL LAYERS

- 1. DOMAIN (CENTER)**
Core business rules
 - Entities
 - Business rules
 - Core logic
- 2. APPLICATION**
Use cases / services / CQRS
 - Application services
 - Interfaces
 - Orchestration
- 3. INFRASTRUCTURE**
Database / APIs / external services
 - Data access
 - External integrations
 - Frameworks
- 4. PRESENTATION**
Controllers / UI
 - Web API / MVC
 - Handles requests & responses

CENTER = DOMAIN LAYER

The Domain layer contains:

- ✓ Entities
- ✓ Business rules
- ✓ Core logic

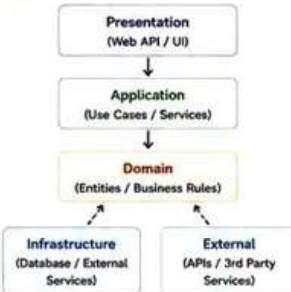


And it should NOT depend on:

- ✗ Database
- ✗ UI
- ✗ External frameworks

Domain is the heart of your application.

REAL DEPENDENCY FLOW



Infrastructure also depends on Domain.

ONION vs CLEAN ARCHITECTURE

ONION ARCHITECTURE



- Focuses heavily on dependency direction
- Domain at the center
- Simpler and more pragmatic
- Looks like an onion 🍷

CLEAN ARCHITECTURE



- Broader architecture principles & boundaries
- Focus on independence from frameworks
- More rules, boundaries and abstractions
- More of a philosophy than a shape

Both aim for the same goal: Independent, maintainable, testable code.

WHY COMPANIES USE IT

- ✓ Easier testing
- ✓ Flexible infrastructure
- ✓ Independent business logic
- ✓ Better maintainability
- ✓ Supports long-term growth



MISTAKE I MADE

Creating too many layers for small projects 🤔

Result:

- ✗ Complexity increased
- ✗ Development slowed



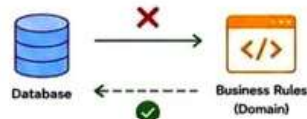
REAL UNDERSTANDING

Architecture should solve problems
NOT create unnecessary complexity.

💡 Start simple. Add layers only when you actually need them.

KEY INSIGHT

Database should depend on business rules,
NOT the other way around! 🔄



HOW IT WORKS (FLOW)



DO'S

- ✓ Keep Domain layer independent
- ✓ Depend on abstractions (Interfaces)
- ✓ Use Dependency Injection
- ✓ Write testable business logic
- ✓ Keep layers focused & small

DON'TS

- ✗ Don't let Domain depend on DB/UI
- ✗ Don't expose internal models
- ✗ Don't put business logic in Controllers
- ✗ Don't create layers you don't need
- ✗ Don't break dependency rule

COMMON PATTERNS USED

- Repository Pattern
- Unit of Work
- Dependency Injection
- Domain-Driven Design (DDD)
- CQRS Pattern

TOOLS & TECHNOLOGIES (EXAMPLE)

- ASP.NET Core (API)
- Entity Framework Core
- MediatR (CQRS)
- AutoMapper
- xUnit / NUnit (Testing)

KEY TAKEAWAY

Onion Architecture keeps business logic at the center and protects it from external changes.
Strong core. Flexible outer layers.

SCENARIO FOR YOU

If your database changes tomorrow, should your business logic really care? 🤔

UP NEXT
Day 40 - Vertical Slice Architecture in .NET
Organizing features differently

REMEMBER
Good architecture is not about more layers. It's about the right layers for the right problems.

DAY 40

VERTICAL SLICE ARCHITECTURE

in .NET

Organizing features differently

THE REAL SCENARIO

Your project structure looks like this:

- Controllers/
- Services/
- Repositories/
- DTOs/
- Validators/

At first: Looks organized 😊

But later:

- ❌ One feature touches MANY folders
- ❌ Hard navigation
- ❌ Features become scattered
- ❌ Hard to maintain & scale 😞



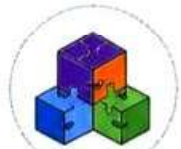
WHAT IS VERTICAL SLICE ARCHITECTURE?

Instead of organizing by technical layers...

You organize by FEATURES.

Each feature contains everything it needs:

- ✔ Request
- ✔ Handler
- ✔ DTO
- ✔ Validation
- ✔ Mapping
- ✔ Business Logic



Together in ONE place.

High cohesion. Low coupling.

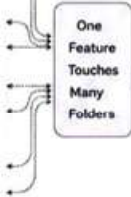
WHY IT MATTERS

- 🔍 Easier feature navigation
Everything related is together
- 🔧 Better maintainability
Change in one place, not many
- 🔗 Less coupling
Features are independent
- 📊 Cleaner scaling
Add new features without affecting others
- 👥 Better developer experience
Faster development & onboarding

TRADITIONAL LAYER STRUCTURE

Code is spread across layers

- Controllers/
 - UserController.cs
 - OrderController.cs
- Services/
 - UserService.cs
 - OrderService.cs
- Repositories/
 - UserRepository.cs
 - OrderRepository.cs
- DTOs/
 - UserDto.cs
 - OrderDto.cs
- Validators/
 - CreateUserValidator.cs
 - CreateOrderValidator.cs



VS

VERTICAL SLICE STRUCTURE

Everything related stays together

- Features/
 - Users/
 - CreateUser/
 - CreateUserCommand.cs
 - CreateUserHandler.cs
 - CreateUserValidator.cs
 - CreateUserDto.cs
 - CreateUserMapping.cs
 - GetUser/
 - GetUserQuery.cs
 - GetUserHandler.cs
 - GetUserDto.cs
 - Orders/
 - CreateOrder/
 - GetOrder/
 - ...



TYPICAL SLICE (EXAMPLE)

Example: CreateUser

- 📄 Command / Query
Represents the request (CreateUserCommand.cs)
- ⚙️ Handler
Contains the business logic (CreateUserHandler.cs)
- ✅ Validator
Validates the request (CreateUserValidator.cs)
- 📄 DTO
Request & Response models (CreateUserDto.cs)
- 🔄 Mapping (Optional)
Mapping logic if needed (CreateUserMapping.cs)

Everything a feature needs, lives together.
Easy to find, easy to maintain.

COMMONLY USED WITH

- ✔ CQRS (Command Query Responsibility Segregation)
- ✔ MediatR
- ✔ Clean Architecture
- ✔ Minimal APIs



MISTAKE I MADE

Trying to force huge abstractions early 😞
But Vertical Slices actually helped simplify things.



REAL UNDERSTANDING

Features are more important than technical folders.

- ✔ Users care about features working
- ❌ Not about perfect folder separation

WHEN NOT TO USE

- ❌ Tiny / simple projects
- ❌ Very small teams
- ❌ Apps with very few features
- ❌ Overkill adds unnecessary complexity

KEY INSIGHT

Keep related code together.
Optimize for maintainability and feature development.

🎯 High Cohesion + Low Coupling = Scalable & Maintainable Code

KEY BENEFITS

- 🎯 Focused Development
- 🚀 Faster Delivery
- 🐛 Easier Testing
- 📄 Less Merge Conflicts
- 🛡️ Long-term Maintainability

EXAMPLE PROJECT STRUCTURE

- src/
 - Features/
 - Users/
 - CreateUser/
 - GetUser/
 - Orders/
 - CreateOrder/
 - GetOrder/
 - Shared/
 - CommonDtos/
 - Mappings/
 - Extensions/

- ✔ Each feature inside its own folder
- ✔ Easy to locate related code
- ✔ Shared things go to Shared/

HOW IT WORKS (FLOW)



Each slice handles one feature end-to-end.
Easy to understand. Easy to extend.



TAKEAWAY

Vertical Slice Architecture organizes code around features, not layers. It reduces scattered logic and improves developer productivity.



SCENARIO FOR YOU

If implementing one feature requires touching 10 folders... is the structure really helping? 🤔



UP NEXT

Day 41 - SQL Basics for Backend Developers
Understanding relational databases



REMEMBER

Good architecture is not about patterns. It's about solving real problems simply.

#dotnet

#csharp

#BackendDevelopment

#webapi

#softwareengineering

#100DaysOfCode

#LearningInPublic

#BackendJourney

#SrinijaBuilds



DAY 41



SQL BASICS

for Backend Developers

Understanding relational databases

THE REAL SCENARIO

Your API works perfectly...

But suddenly:

- ✗ Slow queries
- ✗ Duplicate data
- ✗ Bad joins
- ✗ Performance issues



That's when I realized:

Backend development is
NOT only APIs.
Understanding databases is
equally important.



WHAT IS SQL?

SQL = Structured Query Language

Used to:

- ✓ Store data
- ✓ Retrieve data
- ✓ Update data
- ✓ Manage relational databases

Application
(Your API)

Database

SQL

COMMON DATABASE CONCEPTS



TABLE

Stores related data

Example:

Users Orders Products



ROW

Single record inside a table

Example:

Id: 1, Name: Srinija, Email: srinija@example.com



COLUMN

Defines data fields

Example:

Id Email CreatedDate IsActive

BASIC SQL OPERATIONS

1 SELECT (Read)

Retrieve data from one or more tables.

```
SELECT *
FROM Users;
```

✦ Retrieves all users.

2 WHERE (Filter)

Filter data based on conditions.

```
SELECT *
FROM Users
WHERE IsActive = 1;
```

✦ Retrieves active users.

3 INSERT (Create)

Insert new data into a table.

```
INSERT INTO Users
(Name, Email)
VALUES
('Srinija', 'srinija@example.com');
```

✦ Adds a new user.

4 UPDATE (Modify)

Update existing data.

```
UPDATE Users
SET Name = 'Sree'
WHERE Id = 1;
```

✦ Updates user name where Id = 1.

5 DELETE (Remove)

Delete data from a table.

```
DELETE FROM Users
WHERE Id = 1;
```

✦ Deletes user where Id = 1.

6 COUNT (Aggregate)

Get total number of records.

```
SELECT COUNT(*)
FROM Users
WHERE IsActive = 1;
```

✦ Counts active users.

WHY SQL MATTERS FOR BACKEND DEVELOPERS



Better Query Optimization



Easier Debugging



Better Database Design



Improved API Performance



More Control Over Data

MISTAKE I MADE

Relying completely on ORM-generated queries

Result:

- ✗ Slow APIs
- ✗ Unoptimized queries
- ✗ Hard debugging



REAL UNDERSTANDING

ORMs are helpful

BUT

SQL knowledge is still essential.



Use ORMs for productivity.

Use SQL knowledge for performance & control.

KEY INSIGHT

Bad database design can break even a good backend architecture.



COMMON SQL DATA TYPES (EXAMPLES)

Data Type	Used For
INT	Whole numbers
VARCHAR(n)	Text data
DATETIME	Date and time
BIT	True / False (0 or 1)
DECIMAL(p,s)	Decimal numbers
TEXT	Large text data

SAMPLE TABLE

Users Table				
Id (INT)	Name (VARCHAR)	Email (VARCHAR)	IsActive (BIT)	CreatedDate (DATETIME)
1	Srinija	srinija@example.com	1	2024-05-18 09:00:00
2	Raju	raju@example.com	1	2024-05-18 10:15:00
3	Anu	anu@example.com	0	2024-05-18 11:30:00
...	...	...	...	...

GOOD PRACTICES

- ✓ Always use meaningful table & column names
- ✓ Normalize your database design
- ✓ Index important columns
- ✓ Avoid SELECT * in production
- ✓ Analyze and optimize slow queries

WHEN TO WRITE RAW SQL?

- ✓ Complex queries
- ✓ Performance-critical operations
- ✓ Bulk data operations
- ✓ Reporting & analytics
- ✓ When ORMs are not enough

TOOLS I USE

- SSMS (SQL Server Management Studio)
- Azure Data Studio
- DBeaver
- TablePlus
- dbdiagram.io (Diagram)

REMEMBER

Good APIs + Good Databases
= Scalable Applications

Understand SQL.
Build Better Backends.

TAKEAWAY

Backend developers should understand SQL fundamentals deeply. Database performance directly affects API performance.

SCENARIO FOR YOU

If your API is slow, how would you know whether the issue is in code or the database query?

UP NEXT

Day 42 - SQL Joins Explained
Understanding how tables connect

KEEP PRACTICING

Practice writing queries. Understand execution. Build high-performance applications.

#dotnet

#csharp

#backenddevelopment

#webapi

#softwareengineering

#100DaysOfCode

#LearningInPublic

#SQL

#database

#SrinijaBuilds

DAY 42

SQL JOINS EXPLAINED



Understanding how tables connect

THE REAL SCENARIO

You have:

- Users table
- Orders table

Now you want:

"Show all users with their orders"

How do databases connect this data? 🤔

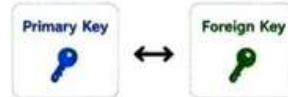
That's where SQL JOINS come in.



WHAT IS A JOIN?

- ✓ JOIN combines data from multiple tables
- ✓ Based on related columns

Usually through:



EXAMPLE TABLES

Users

Id	Name
1	Srinija
2	Rahul

Orders

Id	UserId	Product
1	1	Laptop
2	1	Mouse

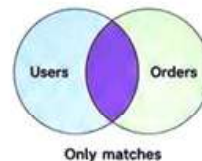
Orders.UserId is a Foreign Key that references Users.Id

1 INNER JOIN

Returns only matching records.

Non-matching records are excluded.

```
SELECT Users.Name, Orders.Product
FROM Users
INNER JOIN Orders
ON Users.Id = Orders.UserId;
```



Only matches

Result

Name	Product
Srinija	Laptop
Srinija	Mouse

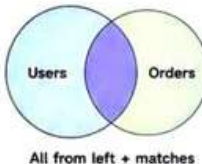
- ✓ Srinija + Laptop
- ✓ Srinija + Mouse
- ✗ Rahul excluded (no order)

2 LEFT JOIN

Returns ALL records from the left table (Users).

Non-matching from right table will be NULL.

```
SELECT Users.Name, Orders.Product
FROM Users
LEFT JOIN Orders
ON Users.Id = Orders.UserId;
```



All from left + matches

Result

Name	Product
Srinija	Laptop
Srinija	Mouse
Rahul	NULL

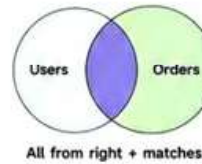
- ✓ Includes Rahul too (with NULL order)

3 RIGHT JOIN

Returns ALL records from the right table (Orders).

Non-matching from left table will be NULL.

```
SELECT Users.Name, Orders.Product
FROM Users
RIGHT JOIN Orders
ON Users.Id = Orders.UserId;
```



All from right + matches

Result

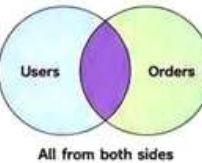
Name	Product
Srinija	Laptop
Srinija	Mouse

- ! Less commonly used in practice.

4 FULL OUTER JOIN

Returns ALL records when there is a match in either left or right table.

```
SELECT Users.Name, Orders.Product
FROM Users
FULL OUTER JOIN Orders
ON Users.Id = Orders.UserId;
```



All from both sides

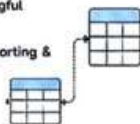
Result

Name	Product
Srinija	Laptop
Srinija	Mouse
Rahul	NULL

- ✓ Includes all users and all orders.

WHY JOINS MATTER

- ✓ Connect relational data
- ✓ Avoid duplicate storage
- ✓ Retrieve meaningful business data
- ✓ Powerful for reporting & analytics



MISTAKE I MADE

Using joins without understanding relationships 🤔

- Result:
- ✗ Duplicate rows
 - ✗ Wrong results
 - ✗ Slow queries



REAL UNDERSTANDING

Understanding table relationships is MORE important than memorizing syntax.



KEY INSIGHT

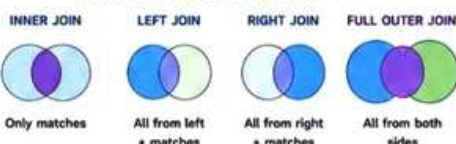
Bad joins can destroy query performance 🚫

Especially with:

- Large tables
- Missing indexes
- Incorrect relationships



JOIN TYPES VISUAL SUMMARY



BEST PRACTICES

- ✓ Always define JOIN conditions correctly
- ✓ Use meaningful table aliases
- ✓ Index your JOIN columns
- ✓ Select only needed columns (avoid SELECT *)
- ✓ Test queries with real data
- ✓ Understand the data before writing the JOIN

COMMON USE CASES

- Users and Orders
- Orders and Products
- Employees and Departments
- Invoices and Payments
- Many more in real applications

TAKEAWAY

SQL joins are how relational databases connect data.
Understanding relationships is the real key.
Good relationships = Accurate results + Better performance

SCENARIO FOR YOU

If your query joins multiple huge tables, how would you keep performance fast? 🤔

UP NEXT

Day 43 - SQL Indexes Explained
Why some queries become fast.

REMEMBER

The power of databases is in relationships.
Master joins, master data! 🏆

#dotnet

#csharp

#backenddevelopment

#sql

#database

#100DaysOfCode

#LearningInPublic

#BackendJourney

#SrinijaBuilds



DAY 43



SQL INDEXES EXPLAINED

Why some queries become fast



THE REAL SCENARIO

Your table has 10 million rows.
And this query becomes VERY slow 🐢

```
SELECT * FROM Users
WHERE Email = 'test@gmail.com';
```

Why does this happen?

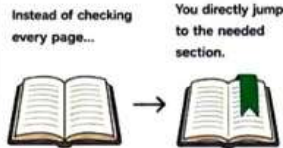
That's where SQL Indexes matter.

Without indexes,
databases may scan
EVERYTHING.



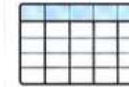
WHAT IS AN INDEX?

- ✓ An index helps SQL Server locate data faster.
- ✓ Similar to an index in a book 📖



WITHOUT vs WITH INDEX

WITHOUT INDEX

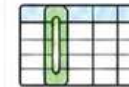


Database scans:

Row 1
Row 2
Row 3
...
Millions of rows 🐢

✗ Slow query performance

WITH INDEX



Use index

Jump directly to
matching row ✓

✓ Faster query execution

CREATING AN INDEX

Creates an index on the Email column of Users table.

```
CREATE INDEX IX_Users_Email
ON Users(Email);
```

Users Table				
Id (PK)	Name	Email	CreatedDate	...
1	Srinja	srinja@example.com	2024-05-18	...
2	Rahul	rahul@example.com	2024-05-18	...

Now SQL Server can find rows using the index instead of scanning the whole table.

COMMON COLUMNS TO INDEX

- ✓ Frequently searched columns 🔍
- ✓ JOIN columns 🔗
- ✓ Foreign keys 🔑
- ✓ WHERE clause columns 🔍
- ✓ Columns used in ORDER BY / GROUP BY ⬆
- ✓ Unique columns 🛡

IMPORTANT UNDERSTANDING

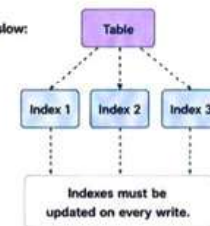
✓ Indexes improve READ performance.

BUT...

Too many indexes can slow:

- ✗ INSERT
- ✗ UPDATE
- ✗ DELETE

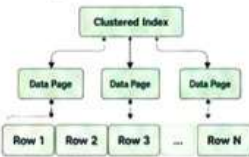
Because indexes also need updating.



CLUSTERED vs NON-CLUSTERED INDEX

CLUSTERED INDEX

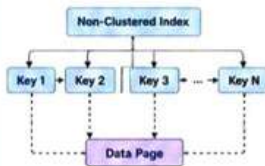
Organizes the actual table data.
Usually the PRIMARY KEY.



Only one clustered index per table.

NON-CLUSTERED INDEX

Separate structure that points to the actual data.
Used for faster lookups.



Many non-clustered indexes per table.

MISTAKE I MADE

Adding indexes everywhere 🤡

Result:

- ✗ Slower writes
- ✗ Extra storage usage
- ✗ Unnecessary complexity



REAL UNDERSTANDING

Good indexing is about BALANCE.

Not: "Index every column."

Too Few
Indexes
= Slow Reads



Too Many
Indexes
= Slow Writes

KEY INSIGHT

Bad indexing can destroy database performance 📉

Even if:

- ✓ API code is good
- ✓ Architecture is clean
- ✓ Hardware is powerful



BENEFITS OF PROPER INDEXING

- ✓ Faster query performance
- ✓ Reduced CPU usage
- ✓ Better user experience
- ✓ Supports scalability
- ✓ Less I/O operations

TYPES OF INDEXES (QUICK VIEW)

Index Type	Description
Clustered Index	Sorts and stores the actual data rows.
Non-Clustered Index	Stores index separately, points to data rows.
Unique Index	Ensures all values in index are unique.
Composite Index	Index on multiple columns.
Filtered Index	Index on a subset of rows (with condition).

BEST PRACTICES

- ✓ Index only what you need.
- ✓ Analyze query patterns before indexing.
- ✓ Use composite indexes wisely.
- ✓ Avoid indexing very small tables.
- ✓ Monitor and remove unused indexes.
- ✓ Check and update query execution plans.



WHEN TO CHECK FOR MISSING INDEXES?

- ➔ Slow queries in production
- ➔ High CPU or I/O usage
- ➔ Full table scans in execution plan
- ➔ High duration of SELECT queries
- ➔ Large tables with frequently used filters or joins



TOOLS TO HELP

- ➔ SQL Server Management Studio (SSMS)
- ➔ Execution Plan (Ctrl + M)
- ➔ DMVs (sys.dm_exec_query_stats)
- ➔ sp_Blitzindex (Community tool)
- ➔ Query Store



REMEMBER

Indexes are not free.
Use them smartly to get maximum performance!



TAKEAWAY

Indexes help databases find data faster.
Proper indexing is critical for scalable applications.



SCENARIO FOR YOU

If your query becomes slow on millions of rows, how would you investigate whether indexing is missing?



UP NEXT

Day 44 - Stored Procedures in SQL Server - when and why they are used



KEEP PRACTICING

The more you practice SQL, the better backend developer you become! 🚀

#dotnet

#csharp

#backenddevelopment

#sql

#database

#100DaysOfCode

#LearningInPublic

#BackendJourney

#SrinjaBuilds



DAY 44



STORED PROCEDURES IN SQL SERVER

When and why they are used



THE REAL SCENARIO

Your application repeatedly runs:

- Complex queries
- Reports
- Business calculations

Should all that SQL logic stay inside application code? 🤔

That's where
Stored Procedures come in.

Earlier, I thought Stored Procedures are outdated
But many large applications still rely on them heavily.

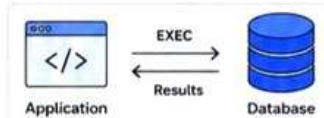


WHAT IS A STORED PROCEDURE?

A precompiled SQL block stored inside the database.

It can:

- ✓ Accept parameters
- ✓ Execute queries
- ✓ Return results
- ✓ Contain business logic



WHY COMPANIES USE STORED PROCEDURES

- Centralized SQL logic
- Better security control
- Reusable queries
- Reduced network traffic
- Sometimes better performance

1 SIMPLE EXAMPLE

```
CREATE PROCEDURE GetUsers
AS
BEGIN
    SELECT * FROM Users;
END
```

EXECUTE PROCEDURE
EXEC GetUsers;

2 PROCEDURE WITH PARAMETERS

```
CREATE PROCEDURE GetUserById
    @Id INT
AS
BEGIN
    SELECT * FROM Users
    WHERE Id = @Id;
END
```

EXECUTE WITH PARAMETER
EXEC GetUserById @Id = 1;

3 PROCEDURE WITH MULTIPLE PARAMETERS

```
CREATE PROCEDURE GetOrders
    @UserId INT,
    @Status NVARCHAR(20)
AS
BEGIN
    SELECT * FROM Orders
    WHERE UserId = @UserId
    AND Status = @Status;
END
```

EXECUTE
EXEC GetOrders
@UserId = 1, @Status = 'Shipped';

4 PROCEDURE RETURNS RESULTS

```
CREATE PROCEDURE GetUserCount
AS
BEGIN
    SELECT COUNT(*) AS TotalUsers
    FROM Users;
END
```

EXECUTE
EXEC GetUserCount;

TotalUsers
1250

IMPORTANT UNDERSTANDING

Stored Procedures are **NOT** automatically faster.

Performance depends on:

- ✓ Query quality
- ✓ Indexing
- ✓ Execution plans



MISTAKE I MADE

Putting TOO MUCH business logic inside procedures 🤔

Result:

- ✗ Hard maintenance
- ✗ Difficult debugging
- ✗ Logic split between app & DB



REAL UNDERSTANDING

Stored Procedures are useful for:

- ✓ Complex reporting
- ✓ Heavy SQL operations
- ✓ Enterprise database systems

BUT...

- ✗ Not every CRUD query needs one.

STORED PROCEDURE vs ORM

ORM (e.g., Entity Framework)

- Easier development
- Cleaner app code
- Database agnostic
- Good for simple CRUD operations



STORED PROCEDURE

- More database control
- Better for complex SQL
- Reusable & secure
- Useful for reporting & heavy operations

WHEN TO USE STORED PROCEDURES?



Complex Reports



Heavy Calculations



Business Logic at DB level



Security & Permission Control



Performance Critical Operations

KEY INSIGHT

Balance matters more than blindly choosing one approach. ⚖️

Use the right tool for the right job.



TAKEAWAY

Stored Procedures centralize SQL operations inside the database. Use them where they genuinely add value.

BENEFITS SUMMARY

- ✓ Centralized & consistent SQL
- ✓ Better security
- ✓ Reusability
- ✓ Reduced network traffic
- ✓ Potential performance gains

POTENTIAL DOWNSIDES

- ✗ Harder to debug sometimes
- ✗ Versioning & deployment complexity
- ✗ Can lead to tight coupling
- ✗ Not always faster

BEST PRACTICES

- ✓ Keep procedures small & focused
- ✓ Use meaningful names
- ✓ Handle exceptions properly
- ✓ Use parameters (avoid hardcoding)
- ✓ Review execution plans regularly

TOOLS I USE

- SSMS (SQL Server Management Studio)
- Azure Data Studio
- Execution Plan (Ctrl + M)
- Query Store

REMEMBER

Good SQL + Good Indexes + Good Procedures = High Performance Applications

SCENARIO FOR YOU

If a payment process updates multiple tables, how would you ensure partial updates don't corrupt the database?



UP NEXT

Day 45 - Transactions in SQL
Keeping data consistent and reliable.



KEEP PRACTICING

Practice writing stored procedures. Understand use cases. Build real-world projects.



LET'S BUILD TOGETHER

Learning in public.
Growing together.
#SrinijaBuilds

#dotnet #sql #backenddevelopment #webapi #softwareengineering #100DaysOfCode #LearningInPublic #sqlserver #database

DAY 45



TRANSACTIONS IN SQL

Keeping data consistent and reliable



THE REAL SCENARIO

Imagine this banking operation:

- 1 Deduct money from Account A
- 2 Add money to Account B

What if:

- ❌ Money gets deducted BUT
- ❌ Transfer fails before adding to Account B? 😞

That's where SQL Transactions become critical.



WHAT IS A TRANSACTION?

A transaction is a group of SQL operations that execute together as ONE unit.



EXAMPLE TRANSACTION

```
BEGIN TRANSACTION;
UPDATE Accounts
SET Balance = Balance - 1000
WHERE Id = 1;
UPDATE Accounts
SET Balance = Balance + 1000
WHERE Id = 2;
COMMIT;
```

IF SOMETHING FAILS

ROLLBACK;

Database returns to previous state.

ACID PROPERTIES

Transactions follow ACID principles

A

ATOMICITY

All operations succeed or fail together.



C

CONSISTENCY

Database remains valid before and after transaction.



I

ISOLATION

Transactions don't interfere with each other incorrectly.



D

DURABILITY

Committed data stays saved permanently.



MISTAKE I MADE

Forgetting transactions in multi-step operations 😞

Result:

- ❌ Partial updates
- ❌ Inconsistent data
- ❌ Difficult debugging



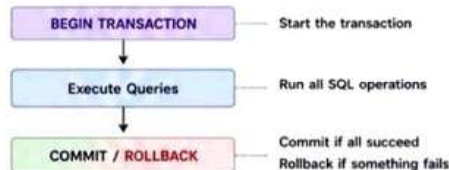
REAL UNDERSTANDING

Transactions are ESSENTIAL for:

- ✔ Payments
- ✔ Order processing
- ✔ Inventory systems
- ✔ Banking operations



COMMON SQL FLOW



IMPORTANT UNDERSTANDING

Long-running transactions can:

- ❌ Lock tables
- ❌ Reduce performance
- ❌ Cause deadlocks



So transactions should:
Stay short & focused

GOOD EXAMPLE (WITH TRANSACTION)



BAD EXAMPLE (WITHOUT TRANSACTION)



KEY INSIGHT

Data consistency is MORE important than partial success.



BEST PRACTICES



Keep transactions short & simple



Use proper error handling



Index your tables



Avoid user interactions inside transactions



Test failure scenarios

TRANSACTION ISOLATION LEVELS

- READ UNCOMMITTED - Can read uncommitted data
- READ COMMITTED - Default level
- REPEATABLE READ - Prevents non-repeatable reads
- SERIALIZABLE - Highest isolation

Choose the right level based on your requirement.



WHEN NOT TO USE TRANSACTIONS?

- Single simple queries
- Read-only operations
- Non-critical logging operations

Don't use transactions unnecessarily. It can impact performance.



REMEMBER

- Transactions = Reliability
- Consistency = Trust
- Think Before You Commit!



TAKEAWAY

Transactions protect database consistency. Multiple related operations should behave as ONE reliable unit.



SCENARIO FOR YOU

If an order is created but payment fails midway, should the database keep partial data?



UP NEXT

Day 46 - Deadlocks in SQL Server
Why queries sometimes freeze each other



KEEP PRACTICING

The more you practice SQL, the better backend developer you become! 🙌

#dotnet #csharp #backenddevelopment

#sql

#database

#100DaysOfCode

#LearningInPublic

#BackendJourney

#SrinijaBuilds

DAY 46

DEADLOCKS IN SQL SERVER



Why queries sometimes freeze each other

THE REAL SCENARIO

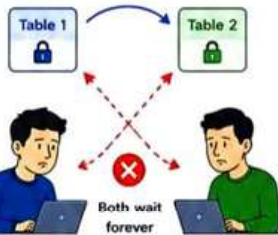
Two users update data at the same time...

User A:

Locks Table 1
Needs Table 2

User B:

Locks Table 2
Needs Table 1



Now both wait forever 😞
That's called a SQL Deadlock.

WHAT IS A DEADLOCK?

A deadlock happens when two transactions wait on each other's locked resources. Neither can continue.

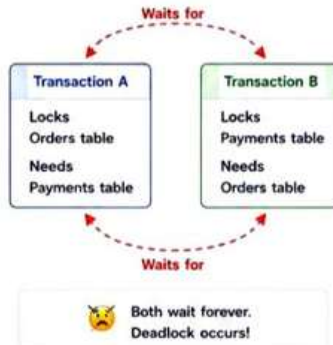
SQL Server then:

Kills one transaction automatically to break the deadlock.



One transaction is chosen as victim and rolled back.

SIMPLE DEADLOCK FLOW



EXAMPLE SCENARIO

Transaction A

```
BEGIN TRANSACTION
UPDATE Orders
SET Status = 'Done'
WHERE Id = 1;
```

Transaction B

```
BEGIN TRANSACTION
UPDATE Payments
SET Status = 'Paid'
WHERE Id = 1;
```

Now if both try accessing each other's tables:



Deadlock possible

WHY DEADLOCKS MATTER

- Failed transactions
- User-facing errors
- Performance issues
- Difficult debugging

MISTAKE I MADE

Keeping transactions open too long 😞

Result:

- More locking
- Higher deadlock chances



REAL UNDERSTANDING

Deadlocks are often caused by:

- Long transactions
- Different table access order
- Missing indexes
- Heavy concurrency



HOW TO REDUCE DEADLOCKS

- Keep transactions short
- Access tables in same order
- Add proper indexes
- Avoid unnecessary locks
- Optimize queries

KEY INSIGHT

Faster queries =
Shorter locks =
Fewer deadlocks



IMPORTANT UNDERSTANDING

Deadlocks are NORMAL in high-concurrency systems.

- Reduce them
- Handle retries properly

NOT:

- Expect zero deadlocks forever.



SQL SERVER BEHAVIOR

When deadlock occurs:

- SQL Server selects a "victim"
- Rolls back one transaction

You may see:

Transaction was deadlocked on lock resources...



DEADLOCK VICTIM SELECTION

SQL Server chooses the victim based on factors like:

- Lower cost to rollback
- Less work done
- Deadlock priority (if set)



DEADLOCK GRAPH (IN SQL SERVER)

SQL Server can show a deadlock graph to help you understand what happened.



How to capture?

- Use SQL Server Profiler / Extended Events
- Check system health or error logs



TAKEAWAY

Deadlocks happen when transactions block each other in a circular way.



Good query design and short transactions reduce deadlocks.

NEXT UP

Day 47 - Normalization in Databases
Reducing duplicate data properly.

SCENARIO FOR YOU

If multiple users update the same tables simultaneously, how would you prevent queries from blocking each other?



BEST PRACTICES

- Keep transactions short & focused
- Use proper indexing
- Access tables in consistent order
- Review & optimize queries regularly

TOOLS TO HELP

- Deadlock graphs
- Extended Events
- DMVs (sys.dm_tran_locks, sys.dm_os_waiting_tasks)
- Query Store

COMMON LOCK TYPES

- S — Shared Lock (Read)
- X — Exclusive Lock (Write)
- U — Update Lock
- Intent Locks (IS, IX, SIX)

RELATED TOPICS

- Transactions
- Isolation Levels
- Locking & Blocking
- Concurrency Control

REMEMBER

Concurrency is powerful, but it must be handled carefully. 🧠

#dotnet

#sql

#backenddevelopment

#database

#100DaysOfCode

#LearningInPublic

#BackendJourney

#SrinijaBuilds

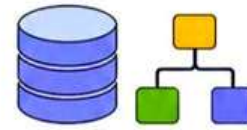


DAY 47



NORMALIZATION IN DATABASES

Reducing duplicate data properly



THE REAL SCENARIO

Imagine storing customer data like this:

OrderId	CustomerName	CustomerPhone
1	Srinija	9999999999
2	Srinija	9999999999
3	Srinija	9999999999
...	...	...

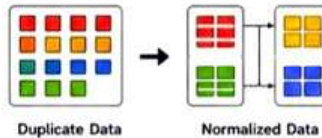
Same customer data repeated everywhere.
Now imagine: Customer changes phone number.
You must update: **✗** Every row manually

That's where
Database Normalization helps.

WHAT IS NORMALIZATION?

A database design technique
used to reduce duplicate data.

- ✓ Better consistency
- ✓ Easier maintenance
- ✓ Cleaner relationships



WHY NORMALIZATION MATTERS

- Reduces duplicate data
- Easier updates
- Better consistency
- Cleaner database structure

1NF FIRST NORMAL FORM (1NF)

Remove repeating groups.
Each column should store atomic values.

✗ Bad:	✓ Better:
Phones 999,888	Phone1 Phone2 999 888

One value per column. No repeating groups.

2NF SECOND NORMAL FORM (2NF)

Remove partial dependency.
Data should depend on
the FULL primary key.

OrderId	ProductId	ProductName
1	101	Laptop

Depends on partial key (OrderId) → Depends on full key (OrderId, ProductId)

All non-key columns must depend
on the full primary key.

3NF THIRD NORMAL FORM (3NF)

Remove transitive dependency.
Non-key columns should depend
ONLY on the primary key.

CustomerId CustomerName CityId	→	CityId CityName State
--------------------------------------	---	-----------------------------

✗ Bad
CityName depends on CityId, not on CustomerId

✓ Good (3NF)
Non-key columns depend only on the primary key.

BETTER DESIGN EXAMPLE

Customers Table

CustomerId	Name	Phone
1	Srinija	9999999999
2	Ravi	8888888888
3	Sneha	7777777777

Orders Table

OrderId	CustomerId	OrderDate
101	1	2024-05-01
102	1	2024-05-05
103	2	2024-05-06

Customer data stored once. Referenced using CustomerId ✓

MISTAKE I MADE

Over-normalizing everything

- Result:
- ✗ Too many joins
 - ✗ Complex queries
 - ✗ Harder reporting sometimes



REAL UNDERSTANDING

Normalization improves consistency.
BUT...

Sometimes controlled
denormalization is useful for:

- ✓ Performance
- ✓ Analytics
- ✓ Reporting systems



KEY INSIGHT

Database design
is always about
BALANCE.



COMMON REAL-WORLD FLOW



Connected using relationships
instead of duplicate data.

WHEN TO CONSIDER DENORMALIZATION

It can be useful in cases like:

- ✗ High read performance is critical
- ✗ Complex reporting / aggregations
- ✗ Data is mostly read, rarely updated
- ✗ Analytics / Data warehouse systems



1 1NF

Atomic values
No repeating groups



2 2NF

Remove partial dependency
Depend on full primary key



3 3NF

Remove transitive dependency
Non-key depends only on PK



4 BCNF

Stronger version of 3NF
Every determinant is a candidate key



BEST PRACTICES

- ✓ Design tables with purpose
- ✓ Use proper primary keys
- ✓ Avoid duplicate data
- ✓ Understand relationships
- ✓ Normalize up to the right level

TOOLS THAT HELP

- ER Diagrams
- Database Design Tools
- Query Analyzer
- SQL Server Management Studio



REMEMBER

- ✗ Consistency > Convenience
- ✗ Good design saves future pain
- ✗ Clean data = Reliable data



TAKEAWAY

Normalization reduces duplicate data
and improves consistency.
Good database design
balances structure and performance.



#dotnet

#sql

#Backenddevelopment

#database

#100DaysOfCode

#LearningInPublic

#BackendJourney

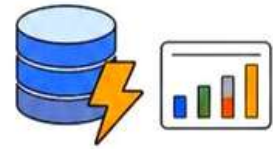
#SrinijaBuilds



DAY 48

DENORMALIZATION IN DATABASES

When duplicate data is actually useful



THE REAL SCENARIO

You normalized your database perfectly...

- ✓ No duplicate data
- ✓ Proper relationships
- ✓ Nice structure

BUT...

Your reports become **VERY** slow 🐢

- ✗ Too many JOINS
- ✗ Complex queries
- ✗ Heavy calculations

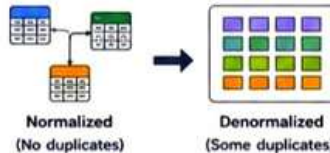
That's where Denormalization comes in.

WHAT IS DENORMALIZATION?

Denormalization means intentionally adding duplicate / precomputed data.

Goal: Improve

- ✓ Read performance
- ✓ Reporting speed
- ✓ Query simplicity



WHY DENORMALIZATION MATTERS

- ⌚ Faster reporting
- 🔗 Reduced joins
- 📊 Better analytics performance
- </> Simpler read queries

NORMALIZED EXAMPLE

Customers Table		Orders Table	
CustomerId	Name	OrderId	CustomerId
1	Srinija	101	1
2	Rohan	102	2
3	Sneha	103	1
...	...	...	...

To get customer name for orders, we need a JOIN.

DENORMALIZED EXAMPLE

Orders Table (Denormalized)			
OrderId	CustomerId	CustomerName	OrderDate
101	1	Srinija	2024-05-01
102	2	Rohan	2024-05-02
103	1	Srinija	2024-05-03
...	...	...	...

- ⚡ Faster reads
- ✗ But duplicate data exists.

IMPORTANT UNDERSTANDING

Denormalization improves READ performance BUT... it increases:

- ✗ Duplicate data
- ✗ Update complexity
- ✗ Risk of inconsistency



MISTAKE I MADE

Trying to optimize too early 🤔

Result:

- ✗ Messy database
- ✗ Data inconsistencies
- ✗ Hard maintenance



REAL UNDERSTANDING

- 1 Design properly
- 2 Normalize correctly

THEN:

Denormalize ONLY where needed.



COMMON REAL-WORLD USAGE

- ✓ Reporting systems
- ✓ Analytics dashboards
- ✓ Data warehouses
- ✓ High-read applications



NORMALIZATION vs DENORMALIZATION

NORMALIZATION

- ✓ High consistency
- ✓ Cleaner structure
- ✓ Easier updates
- ✓ Less storage redundancy



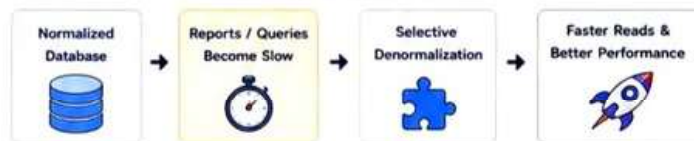
DENORMALIZATION

- ✓ Faster reads
- ✓ Better reporting
- ✓ Less complex queries
- ✓ More storage used



VS

REAL-WORLD FLOW



Denormalize selectively. Not everywhere!

KEY INSIGHT

Performance optimization is always about **TRADEOFFS**.
Not "one perfect design for everything."



WHEN TO DENORMALIZE?

- ✓ When read performance is critical
- ✓ When reports require many joins
- ✓ When data is mostly read, rarely updated
- ✓ When using analytics / reporting systems

WHEN NOT TO DENORMALIZE?

- ✗ When data changes frequently
- ✗ When consistency is very important
- ✗ When storage is limited
- ✗ When unnecessary complexity increases



BEST PRACTICES

- ✓ Normalize first
- ✓ Denormalize based on real needs
- ✓ Document duplicate data clearly
- ✓ Keep data in sync (ETL, Triggers, Jobs)
- ✓ Review and optimize regularly



EXAMPLE: DENORMALIZED REPORTING TABLE

Date	TotalOrders	TotalRevenue	CustomerName
2024-05-01	120	12500	Srinija
2024-05-02	95	9800	Rohan
2024-05-03	150	16600	Sneha
...	...	...	...

Pre-calculated data for fast reporting.

THINK BEFORE YOU DENORMALIZE

Denormalization is powerful. Use it carefully and only when real performance needs exist.



TAKEAWAY

Denormalization intentionally duplicates data for performance benefits. Use it carefully and only when real performance needs exist.

SCENARIO FOR YOU

If your reports require 10+ joins and become very slow, would controlled duplicate data sometimes be acceptable?



#dotnet

#sql

#backenddevelopment

#database

#100DaysOfCode

#LearningInPublic

#BackendJourney

#SrinijaBuilds



DAY 49



EXECUTION PLANS IN SQL SERVER

Understanding how queries actually run



THE REAL SCENARIO

Your query looks simple...

```
SELECT *
FROM Orders
WHERE CustomerId = 10;
```

You already:

- ✓ Added indexes
- ✓ Optimized code

Still slow on production data? 😞

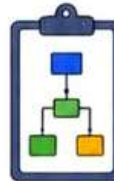
That's where
Execution Plans become important.

WHAT IS AN EXECUTION PLAN?

An execution plan shows how SQL Server executes a query step-by-step.

It helps identify:

- ✓ Table scans
- ✓ Index usage
- ✓ Joins
- ✓ Expensive operations



COMMON EXECUTION FLOW



WHY EXECUTION PLANS MATTER



- ✓ Find slow operations
- ✓ Detect missing indexes
- ✓ Improve query performance
- ✓ Understand SQL behavior

COMMON EXPENSIVE OPERATION

TABLE SCAN

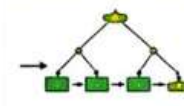
SQL Server checks the entire table.



Very slow on large tables 😞

INDEX SEEK

SQL Server jumps directly to matching rows using index.



Much faster! 🚀

MISTAKE I MADE

Looking only at query syntax 😞

Result:

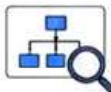
- ✗ Missed actual bottlenecks
- ✗ Slow production queries



REAL UNDERSTANDING

Two queries that LOOK similar can perform VERY differently.

Execution plan reveals what SQL Server is REALLY doing.



COMMON THINGS TO CHECK

- ✓ Table Scan vs Index Seek
- ✓ Expensive Joins
- ✓ Missing Indexes
- ✓ Sort Operations
- ✓ High-Cost Operators
- ✓ Implicit Conversions

IMPORTANT UNDERSTANDING

Query cost percentages are ESTIMATES, NOT absolute truth 🙅

Always combine:

- ✓ Execution plans
- ✓ Real query timing
- ✓ Actual workload analysis



ESTIMATED vs ACTUAL PLAN

ESTIMATED PLAN

SQL Server prediction



VS

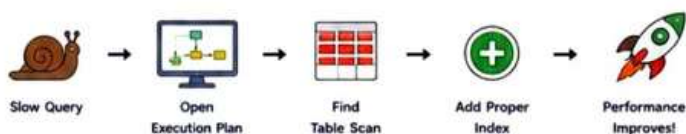
ACTUAL PLAN

What REALLY happened during execution



Actual plans are usually more useful.

EXAMPLE INVESTIGATION FLOW



★ Execution plans help you go from SLOW to FAST! 🚀

NORMALIZATION vs DENORMALIZATION (QUICK RECAP)

NORMALIZATION

- ✓ High consistency
- ✓ Less duplicate data
- ✓ Cleaner structure
- ✓ Better for writes

VS

DENORMALIZATION

- ✓ Faster reads
- ✓ Better reporting
- ✓ Duplicate data allowed
- ✓ Better for analytics

EXECUTION PLAN EXAMPLE (Actual Plan Snippet)



(Costs are relative estimates)

KEY INSIGHT

Good SQL tuning starts with understanding execution plans.

Not random optimization guesses! 😞

BEST PRACTICES

- ✓ Always check execution plan for slow queries
- ✓ Prefer actual execution plan
- ✓ Look for table scans
- ✓ Check missing index suggestions
- ✓ Review and test after changes

TOOLS TO VIEW PLANS

- SSMS - Include Actual Execution Plan (Ctrl + M)
- SSMS - Include Estimated Execution Plan (Ctrl + L)
- Azure Data Studio
- Query Store & DMVs



REMEMBER

- ★ The optimizer chooses the best plan (most of the time).
- ★ Good stats = Good plans
- ★ Data distribution impacts plans

TAKEAWAY

Execution Plans are one of the most powerful tools for performance tuning in SQL Server.



#dotnet

#sql

#backenddevelopment

#database

#100DaysOfCode

#LearningInPublic

#BackendJourney

#SrinijaBuilds

